Addis Ababa Science and Technology University

College of Engineering

Department of Software Engineering

MEKARI: A Collaboration Platform

Addis Ababa Science and Technology University

College of Engineering

Department of Software Engineering

Title: MEKARI: A Collaboration Platform

Group Members:	ID
1. Betselot Tesfa	ETS0327/14
2. Edom Mulugeta	ETS0503/14
3. Gelila Nebiyu	ETS0690/14
4. Meklit Habtamu	ETS1030/14
5. Rafia Kedir	ETS1329/14

Advisor Name: Biruk G. Signature_____

[December, 2025]

Acknowledgement

We, the members of the MEKARI project extend our deepest gratitude to all those who have played a pivotal role in the successful completion of this capstone project. As a collaborative endeavor in software engineering at Addis Ababa Science and Technology University (AASTU), MEKARI embodies the spirit of student teamwork, innovation and peer support that defines our academic journey.

First and foremost, we sincerely thank our advisor, Biruk G., whose unwavering guidance, insightful feedback and encouragement were instrumental in shaping our vision and refining our approach. His expertise in agile methodologies and real-time systems not only steered us through challenges but also inspired us to foster a platform that promotes interdisciplinary collaboration among students and professionals alike.

We are profoundly grateful to the College of Engineering and the Department of Software Engineering at AASTU for providing the academic foundation, resources and environment that nurtured our growth. The university's commitment to hands-on capstone projects has been invaluable, enabling us to transform theoretical knowledge into a practical solution for technical peer support. This project would not have been possible without the collective efforts of our team. Each member brought unique strengths creating a synergy that exemplifies student collaboration. We celebrate the late-night sprints, brainstorming sessions and mutual encouragement that turned obstacles into opportunities.

Our heartfelt appreciation also goes to our families and friends, whose unwavering support and understanding sustained us through demanding timelines. Special thanks to the pilot testers and stakeholders who provided candid feedback, enriching MEKARI's design.

Finally, we acknowledge the open-source community and referenced works, whose contributions we have built upon with due permission and attribution. This project is a testament to the power of collaborative learning, and we are excited to contribute it back to our peers.

Contents

Acknowledgement I
Lists of Tables, Figures/Illustrations, Plates/Photographs IV
List of Abbreviations, Symbols, Specialized Nomenclature VI
Abstract X
Chapter One 1
Introduction 1
1.1. Statement of the Problem 2
1.2. Objectives 2
1.2.1. General Objective 2
1.2.2. Specific Objectives 2
1.3. Scope and Limitation 3
1.4. Methodology 5
1.5. Plan of Activities 6
1.6. Significance of the Study 7
1.8 Outline of the study 8
Chapter Two 10
Literature Review 10
2.1. Study Related Works 10
2.2.1 Strengths and Contributions in the Literature 12
2.2.2 Gaps and Open Problems 13
2.3. Lessons Learned from Literatures 13
2.4. Comparative Analysis of Related Systems 14
Chapter Three 16
Problem Analysis and Modeling 16
3.1. Existing System and Its Problems 16
3.2. Specifying the Requirements of the Proposed Solution 19
3.3. System Modeling 20
3.3.1. Functional Requirements 21
3.3.2. Non-Functional Requirements (NFRs) 24
3.3.3. MoSCoW Prioritization 27
3.3.5. Dynamic Models of a System 39

3.3.5.1. Sequence Diagrams 39
3.3.5.2. Activity Diagrams 41
3.3.5.3. Class Diagrams 42
3.3.5.4. ERD Diagrams 44
3.5. Model Validation 45
Chapter Four 46
System Design 46
4.1. Overview 46
4.2. Specifying the Design Goals 46
4.3. System Design 48
4.3.1. Proposed Software Architecture 50
4.3.2. Subsystem Decomposition 51
4.3.3. Database Design 53
4.3.4. Deployment Diagram 55
4.3.5. User Interface Design 56
4.4. Verifying the Requirements in the Design 60
Chapter Five 64
System Implementation 64
5.1. Reviewing the Design Solution 64
5.2. Deciding on the Development Tools 66
5.3. Developing the Solution 69
Chapter Six 74
System Evaluation 74
6.1. Preparing Sample Test Plans 74
6.2. Evaluating the Proposed Design and Solutions 78
6.3. Discussing the Results 82
Chapter Seven 85
Conclusions and Recommendations 85
7.1. Conclusion of the Study 85
7.2. Recommendations of the Study 86
References 88
Appendices 89

Lists of Tables, Figures/Illustrations, Plates/Photographs

List of Tables

Table 1: Comparative Analysis 14
Table 2: Current Fragmented Workflow for Technical Support 18
Table 3: MoSCoW Table 27
Table 4: Actor Identification Table 29
Table 5: Use Case: Register and Log In 30
Table 6: Use Case: Manage User Profile and Availability 31
Table 7: Use Case: Post Query with Subject Tagging 32
Table 8: Use Case: Match and Notify Experts 33
Table 9: Use Case: Engage in Real-Time Chat and Live Session 34
Table 10: Use Case: Interact with AI Chatbot and Escalate 34
Table 11: Use Case: Mark Thread as Solved and Log Issue 35
Table 12: Use Case: Participate in Gamification and View Leaderboards 36
Table 13: Use Case: Moderate Content and Manage Community Safety 37
Table 14: Use Case: View Analytics and Trending Topics 37
Table 15: Subsystem Decomposition 51
Table 16: Key Integration Flows 58
Table 17: Requirements Verification Plan 61
Table 18: Core Technology Stack 66
Table 19: Development, Build, and Quality Tools 67
Table 20: Challenges and Strategies 72
Table 21: Classification of Test Types and Tools Employed 75
Table 22: Planned Core Functional Test Cases – Critical Features 79
Table 23: Planned Edge-Case and Negative Test Cases 80
Table 24: Planned Performance and Scalability Metrics 81
Table 25: Planned Security and Accessibility Evaluation 81
Table 26: Metrics 82
Table 27: Demographic Breakdown 91
Table 28: Current Platforms Used 91
Table 29: Biggest Challenges 92

Table 30: Feature Importance (Mean on 1–5 Scale) 92
Table 31: Preferred Connection Methods and Incentives (Multiple Select) 93

List of Figures

Figure 1: Usecase Diagram 38
Figure 2: Sequence Diagram: Marking solved issues 40
Figure 3: Sequence Diagram: Expert escalation 41
Figure 4: Activity Diagram 42
Figure 5: Class Diagrams 42
Figure 6: ERD diagram 44
Figure 7: Database Design 53
Figure 8: Deployment Diagram 55

List of Abbreviations, Symbols, Specialized Nomenclature

Abbreviations and Acronyms

Abbreviation/Acronym	Full Form	Description/Context in MEKARI
AASTU	Addis Ababa Science and Technology University	The academic institution hosting this capstone project, emphasizing collaborative student innovation.
AES	Advanced Encryption Standard	Encryption algorithm (e.g., AES-256) used for securing user data at rest in the platform's backend.
AI	Artificial Intelligence	Core technology for the context-aware chatbot providing personalized troubleshooting and guidance.
API	Application Programming Interface	Interfaces for backend services (e.g., RESTful APIs) and integrations like Google Meet.
CAPTCHA	Completely Automated Public Turing test to tell Computers and Humans Apart	Security mechanism to prevent automated spam during user registration.
CRUD	Create, Read, Update, Delete	Standard operations for managing user profiles, messages, and issue logs in MongoDB.
CSS	Cascading Style Sheets	Styling framework (e.g., Tailwind CSS) for responsive UI design across devices.
CQA	Community Question Answering	Type of platforms (e.g., Stack Overflow) analyzed in the literature review for expert matching.

ERD	Entity-Relationship Diagram	Modeling tool for database design, illustrating relationships between entities like Users and Threads.
FR	Functional Requirement	Specific features the system must perform (e.g., FR-001: User Registration).
GDPR	General Data Protection Regulation	EU privacy framework guiding data minimization and user consent in MEKARI.
HTML	HyperText Markup Language	Markup language for frontend structure in Next.js components.
JS	JavaScript	Programming language for both frontend (Next.js) and backend (Node.js).
LTS	Long-Term Support	Node.js version (e.g., Node.js 20 LTS) selected for stability in development.
NLP	Natural Language Processing	AI subfield enabling the chatbot's context-aware query understanding and responses.
NFR	Non-Functional Requirement	System qualities (e.g., NFR-003: Page Load Time ≤ 3 seconds).
OAuth	Open Authorization	Protocol for secure third-party authentication (e.g., Google/GitHub login).
REPL	Read-Eval-Print Loop	Stateful code execution environment referenced in tool usage for testing.
REST	Representational State Transfer	Architectural style for backend APIs handling HTTP requests.

Scrum	Scrum Framework	Agile methodology with bi-weekly sprints for iterative student collaboration.
SI	Système International d'Unités	International System of Units; used sparingly for metrics (e.g., latency in seconds).
SUS	System Usability Scale	Standardized questionnaire for evaluating user interface intuitiveness (target: ≥80).
TLS	Transport Layer Security	Protocol (e.g., TLS 1.3) for encrypting data in transit via HTTPS.
TDD	Test-Driven Development	Practice applied to critical components like the expert-matching algorithm.
UAT	User Acceptance Testing	Pilot testing phase with 20–30 diverse students to validate platform usability.
UI/UX	User Interface/User Experience	Design principles for intuitive, accessible interfaces supporting collaborative features.
UML	Unified Modeling Language	Diagramming standard for system modeling (e.g., sequence diagrams for real-time chat).
WebRTC	Web Real-Time Communication	Technology for potential future enhancements in native voice/whiteboard sessions.
XSS	Cross-Site Scripting	Web vulnerability mitigated via input sanitization in Express.js middleware.

Symbols

Symbol	Meaning	Context in MEKARI	SI Unit (if applicable)
≤	Less than or equal to	Used in non-functional requirements (e.g., latency ≤ 2 seconds for chat responses).	N/A
≥	Greater than or equal to	Performance thresholds (e.g., test coverage ≥ 80%).	N/A
%	Percent	Metrics like matching accuracy (e.g., 91%) or SUS score (e.g., 85.8%).	N/A
s	Second	Time-based metrics (e.g., response time in seconds).	s (SI base unit for time)
N	Number (of)	Quantitative measures (e.g., N=25 for UAT participants).	N/A

Abstract

In the dynamic realm of technical education and professional growth, students and practitioners in engineering fields face substantial hurdles in securing prompt, effective peer support. Platforms like Stack Overflow offer asynchronous Q&A, Microsoft Teams enables team communication, Discord supports casual voice chats, and AI tools such as ChatGPT provide instant aid yet they remain fragmented, delayed, and deficient in unified real-time collaboration. This leads to extended issue resolution, isolated knowledge sharing, and reduced mentorship, especially in settings like Addis Ababa Science and Technology University (AASTU).

This capstone project counters these issues through MEKARI, a lightweight full-stack web platform designed for interdisciplinary student technical collaboration. Core aims encompass real-time voice and whiteboard integration via Google Meet APIs, a context-aware AI chatbot for tailored troubleshooting, subject-based expert matching, and gamification via points, badges, and leaderboards to boost engagement. Employing agile Scrum with bi-weekly sprints, the architecture utilizes Node.js/Express backend, MongoDB persistence, Redis for messaging, and Next.js/Tailwind CSS frontend for device-agnostic access. Requirements were elicited from 20–30 stakeholders via surveys and interviews, modeled with UML for dynamics, and prototyped iteratively. Functional elements include threaded chats and issue logging; non-functional targets cover ≤2-second latency and WCAG 2.1 AA accessibility. Modular implementation validated features like weighted matching and AI escalation using Cypress/Mocha tests and UAT.

Results validate efficacy: 91% matching accuracy, 88% AI precision in 3.7 seconds, SUS score of 85.8—exceeding peers. Feedback praises live-session fluidity and gamification's motivational impact.

MEKARI's import lies in transforming student collaboration: it curtails delays, builds searchable knowledge repositories, and nurtures mentorship, aligning with AASTU's equitable education goals. As an open-source prototype, it inspires institutional scaling, AI advancements, and bridges gaps in real-time peer support, enriching software engineering education and practice.

Chapter One
Introduction

In the contemporary landscape of science, technology, engineering and mathematics (STEM) education, collaborative learning stands as a cornerstone for innovation and skill development. As global economies increasingly rely on technical expertise to drive progress, fostering effective peer support mechanisms among students has become imperative.

The current state of affairs in technical peer support reveals stark disparities. Students endure heightened academic pressures, with studies indicating that they exhibit a significantly lower propensity to seek help compared to peers in other disciplines often due to stigma, time constraints, and perceived barriers such as lack of resource awareness. Globally, unresolved technical issues exacerbate these challenges; professionals and students alike lose nearly three hours per week navigating technology glitches, translating to substantial productivity deficits. In STEM fields, this inefficiency contributes to broader economic tolls: projections estimate 3.5 million unfilled STEM jobs by the end of 2025, alongside annual losses of up to $480 billion in the United States alone from talent shortages. In higher education, while adoption of online collaboration platforms has surged eLearning uptake increased by 300% between 2019 and 2021, and 86% of institutions now integrate AI tools the hybrid model's permanence is embraced by nearly 70% of administrators, underscoring a shift toward digital augmentation of traditional learning.

Despite these advancements, existing systems fall short in delivering seamless, real-time support tailored to technical disciplines. Platforms like Stack Overflow provide asynchronous Q&A but impose delays averaging hours to days, while tools such as Microsoft Teams and Discord facilitate communication yet lack automated expert matching or integrated visual aids like whiteboards. Standalone AI solutions, including ChatGPT, offer instant responses but isolate users from community validation and mentorship. Attempted remedies, such as institutional peer-mentoring programs, have shown promise reducing therapy sessions by over two per participant in some digital peer support initiatives but remain siloed, underutilized, and insufficiently scalable across diverse student cohorts. These gaps not only prolong problem resolution but also hinder the cultivation of a collaborative ecosystem essential for student success.

This project introduces MEKARI as a student-led response, a lightweight platform designed to bridge these voids through agile, inclusive innovation, empowering AASTU's engineering community and beyond.

1.1. Statement of the Problem

Individuals in diverse technical fields frequently face challenges in obtaining timely, precise, and effective support for complex problems. Current solutions ranging from comprehensive software for remote collaboration to online forums and social media communities are often inefficient, fragmented, or difficult to navigate, resulting in delayed problem resolution and limited opportunities for knowledge exchange and mentorship.

There is a clear need for a streamlined, accessible platform that connects users with relevant technical expertise in real time. Such a platform should enable interactive voice and whiteboard collaboration, incorporate AI-driven guidance for context-sensitive assistance, and foster community engagement through gamification and rewards. Additionally, robust issue tracking would create an evolving knowledge repository to inform future interactions. By integrating these features, the platform aims to enhance the efficiency and quality of technical support, facilitate rapid problem-solving, and promote collaborative learning and professional growth across technical disciplines.

1.2. Objectives

1.2.1. General Objective

To develop an AI-assisted collaboration platform that connects users seeking technical support with experienced mentors and peers, emphasizing real-time interactive collaboration, context-aware AI guidance, and community-driven knowledge sharing across diverse technical disciplines.

1.2.2. Specific Objectives

- Study and analyze existing technical collaboration and support platforms to identify their strengths, weaknesses, limitations, and best practices in areas such as real-time communication, expert matching, and user engagement.

- Elicit and prioritize user requirements through Google Form surveys and interviews targeting students, professionals, and experts from multiple technical fields to ensure the platform addresses real pain points in timely and effective support.
- Design and implement subject tagging and intelligent expert-matching mechanisms to automatically categorize queries and connect users with relevant available mentors or peers.
- Develop a responsive chat interface supporting threaded conversations, code/image attachments, notifications, mentions, and seamless transitions to live sessions.
- Integrate real-time voice communication and interactive whiteboard features to enable collaborative problem-solving and visual explanations during live sessions.
- Incorporate a context-aware AI chatbot that provides personalized troubleshooting guidance, suggestions, and platform navigation support based on conversation context and logged issues.
- Create customizable user profiles allowing individuals to highlight expertise areas, set availability status, and showcase professional/academic backgrounds for efficient matching.
- Design and implement a gamification and reward system including points, badges, leaderboards, and recognition features to motivate active participation, timely responses, and quality contributions.
- Build a repository for logging technical issues, solutions, and interactions to enable search functionality and data-driven recommendations for future queries.
- Ensure responsive and accessible design optimized for multiple devices (desktops, tablets, smartphones) following best practices for usability, navigation, and inclusivity.
- Implement moderation tools, security controls (e.g., authentication, spam prevention), and administrative features to maintain a safe and respectful community environment.
- Develop usage analytics to track trending topics, active contributors and engagement metrics for continuous platform improvement and community insights.

1.3. Scope and Limitation

Scope:

This project focuses on developing Mekari, a lightweight peer-support platform for users across diverse technical fields. The platform facilitates real-time collaboration, knowledge

sharing, and problem-solving through features such as:

- Subject-specific channels: Focused group discussions and one-on-one assistance for technical topics.
- Real-time helper availability: Quickly identify and connect with available experts.
- Threaded conversations and notifications: Organized communication with mentions and alerts.
- Gamification and reputation systems: Badges, rankings, and recognition to encourage active participation.
- Live session integration: Transition from chat to voice or video sessions via platforms like Google Meet.
- AI chatbot assistance: Context-aware guidance, study recommendations, and platform navigation support.
- Voice and whiteboard collaboration: Real-time visual problem-solving for enhanced comprehension.
- Technical issue logging and solution repository: Build an evolving knowledge base to improve support efficiency.

Limitations:

The initial development phase will exclude resource-intensive tools such as code editors, CAD or 3D modeling software, circuit simulators, process simulation modules, automated grading systems, and monetization features like subscriptions or paid coaching.

By focusing on core communication, expert matching, and interactive support, Mekari aims to provide an accessible, efficient, and collaborative environment for technical problem-solving while remaining lightweight and compatible across devices and network conditions.

1.4. Methodology

This project adopts a user-centered, agile development approach to create a lightweight platform that supports real-time technical collaboration and peer mentorship.

Research Design and Data Collection:

Requirements will be gathered through questionnaires and semi-structured interviews from users across multiple technical disciplines. This ensures inclusion of key features such as expert matching, AI-assisted guidance, and voice/whiteboard collaboration.

Development Approach:

The Scrum framework within agile methodology will guide development through biweekly sprints, allowing iterative feature implementation, stakeholder engagement, and rapid refinement based on feedback.

System Design and Implementation:

- Backend: Node.js with Express for RESTful APIs; MongoDB for user profiles, chat data, and technical issue logs.
- Frontend: Next.js with Tailwind CSS for a responsive, accessible interface across desktops, tablets, and smartphones.
- Real-time Collaboration: Google Meet APIs to support live voice communication and whiteboard interactions.
- AI Integration: A context-aware chatbot leveraging NLP frameworks for personalized guidance and study recommendations.

Testing and Evaluation:

- Unit Testing: Cypress for frontend components and Mocha for backend APIs.
- Integration Testing: Postman to validate interoperability of chat, notifications, and live sessions.
- User Acceptance Testing (UAT): Pilot testing with 20–30 diverse users; feedback collected via surveys and interviews.

- Performance Metrics: Quantitative measures include peer-matching accuracy, collaboration latency, and chatbot relevance.
- Qualitative Analysis: Thematic analysis of user feedback to identify usability improvements and enhance platform effectiveness.

This methodology ensures the platform is robust, user-friendly and responsive, fostering efficient technical problem-solving and collaborative learning.

1.5. Plan of Activities

Months 1–2: Requirements Gathering

- Engage stakeholders across technical disciplines to identify user needs.
- Design and distribute targeted questionnaires; conduct semi-structured interviews.
- Compile, analyze, and validate functional and non-functional requirements with stakeholder feedback.

Months 3–4: System Design

- Draft system architecture including backend, frontend, and database schemas for core features such as chat, subject tagging, and peer matching.
- Develop high-fidelity UI/UX prototypes for key interactions, including messaging, AI chatbot engagement, and helper discovery.
- Design data models for user profiles, messages, and interaction logs.
- Conduct usability testing with a small user group; iterate and finalize design documentation.

Months 5–7: Development

- Implement backend APIs, database schemas, and expert matching algorithms; initiate foundational frontend features.
- Integrate advanced communication tools group channels, notifications, live session links and perform internal code reviews and optimizations.

- Complete remaining features including AI chatbot, voice and whiteboard collaboration, and gamification/rewards; finalize system for testing.

Month 8: Testing and Deployment

- Conduct comprehensive unit and integration testing; perform User Acceptance Testing (UAT) with diverse participants.
- Analyze feedback, fix bugs, and optimize system performance.
- Deploy the platform for broad access and implement monitoring mechanisms for ongoing evaluation and iterative improvement.

1.6. Significance of the Study

Individuals in technical fields often face complex challenges where timely peer support is crucial. Existing platforms frequently suffer from delays, disorganization, or overly broad scopes, limiting effective assistance. Mekari addresses this gap by providing a focused, discipline-aware platform that facilitates efficient peer-to-peer collaboration while maintaining simplicity.

Key contributions of the study include:

- Accelerated problem resolution: Intelligent matching connects users with peers possessing relevant expertise, reducing downtime and improving access to solutions.
- Interdisciplinary knowledge exchange: Encourages collaboration across diverse technical domains, fostering innovative problem-solving.
- Mentorship cultivation: Reputation and feedback mechanisms motivate proactive contributions, promoting a supportive learning community.
- Preservation of cumulative knowledge: Threaded discussions and logged interactions create a searchable repository for future users.
- Data-driven insights: Aggregated, anonymized platform data can inform educators and institutions about common challenges, guiding targeted instructional support.

The insights gained from Mekari can inform the development of scalable, discipline-agnostic

learning tools, with future enhancements potentially including institutional authentication, AI-generated summaries, and advanced collaboration features, all while maintaining the platform's mission of providing accessible, timely, and personalized technical support.

1.8 Outline of the study

This study is organized into seven chapters, providing a structured roadmap for understanding the development and evaluation of Mekari, a real-time technical support platform.

Chapter One: Introduction

Introduces the background and context of the study, highlighting challenges faced by individuals in technical fields when accessing timely and relevant support. It defines the problem, outlines objectives, scope, limitations, methodology, project schedule, budget, and significance of the study.

Chapter Two: Literature Review

Reviews existing research, related platforms, and technologies in technical support, collaborative tools, and AI-driven assistance. This chapter identifies gaps and lessons learned that inform the design and implementation of Mekari.

Chapter Three: Problem Analysis and Modeling

Analyzes the limitations of existing systems and specifies requirements for the proposed solution. Includes functional and non-functional requirements, use cases, and dynamic system models such as sequence, activity, state machine, and collaboration diagrams, along with model validation.

Chapter Four: System Design

Describes the overall system design, including proposed software architecture, subsystem decomposition, database design, deployment, and user interface design. This chapter also demonstrates how the design satisfies the specified requirements.

Chapter Five: System Implementation

Covers the development process, reviewing the design solution, selecting development tools, and implementing the platform's features.

Chapter Six: System Evaluation

Presents testing strategies, sample test plans, and evaluation results, including user acceptance, performance metrics, and usability analysis.

Chapter Seven: Conclusions and Recommendations

Summarizes key findings, conclusions drawn from the study, and recommendations for future improvements or research.

This structured approach ensures a comprehensive understanding of the platform's development, from identifying challenges to implementing and evaluating a practical solution that enhances technical collaboration and knowledge sharing.

Chapter Two
Literature Review

Over the last 20 years, technical support and knowledge sharing have transformed dramatically. What began with static mailing lists and forums has evolved into lively, real-time spaces powered by AI and collaborative tools. This literature review serves two main purposes: it traces the evolution of these platforms, highlighting how each innovation tackled (or sometimes created) new challenges, and it builds a framework showing why Mekari is a smart blend of proven strengths that fixes ongoing weaknesses.

A close look at existing studies uncovers a clear pattern: platforms excel in one area but often fall short in others. Stack Overflow nailed structured knowledge storage but skipped real-time teamwork. Tools like Teams and Discord delivered instant chats at the cost of lasting records. AI chatbots offer quick replies, yet they miss deeper context and human touch. This review positions our work amid these partial fixes, explaining how Mekari bridges that tricky balance between machine speed and human insight, immediacy and endurance.

What stands out isn't a checklist of features to copy, but real insights into how technical communities thrive: how trust builds, expertise gets spotted, people step up to help strangers, and platforms stay vibrant long-term.

2.1. Study Related Works

2.1.1 Stack Overflow

Stack Overflow, launched in 2008, changed the game for technical Q&A with strict formatting, tags, voting, and a sharp line between questions and chit-chat. Treude et al. (2011) showed how this setup sparked accidental expertise, where developers uncover their own knowledge by spelling out solutions.

Less discussed is the emotional side: Stack Overflow assumes you're competent. Questions must show research and effort, which respects experts but intimidates newcomers. Technically, its big flaw is the time gap. Answers live forever via search but offer no live help. Vasilescu et al. (2014) noted median response times around 11 minutes for answered questions, but many (especially niche ones) go unanswered.

2.1.2 Discord

Discord's evolution from a gaming-focused platform into a widely used space for technical communities, particularly during the COVID-19 pandemic, provides an important case study in modern collaboration tools. Beyond features such as voice channels, threads, and screen sharing, Discord introduced strong presence awareness: users can see who is online, which encourages spontaneous interaction and informal knowledge exchange.

However, this informality also introduces limitations. Conversations move quickly, valuable technical insights are often buried in long message histories, and search functionality is poorly suited for retrieving detailed or previously discussed solutions. As a result, the same questions frequently reappear, indicating weak knowledge retention.

Microsoft Teams, by contrast, is designed for workplace collaboration and integrates closely with productivity tools such as Microsoft Office. Its structured channels and document integration aim to improve organization. Nevertheless, Wang et al. (2020) identify a phenomenon they describe as collaboration overload, characterized by excessive notifications, fragmented conversations, and frequent context switching, all of which can reduce focus and hinder deep problem-solving.

2.1.3 AI Chatbots

AI-assisted tools such as GitHub Copilot and ChatGPT represent a third wave of technical support systems. These tools combine code generation, contextual awareness, and conversational interfaces to assist developers during problem-solving. Ziegler et al. (2022) report that developers using GitHub Copilot completed tasks up to 55% faster. However, the authors also caution against an illusion of understanding, where confident and fluent AI-generated output may conceal subtle errors, requiring users to develop stronger verification and critical evaluation skills.

AI chatbots transform traditional question–answer interactions into iterative dialogues, enabling rapid exploration of solutions. Despite these advantages, Xu et al. (2023) find that such systems perform poorly on ill-defined problems, edge cases, and domain-specific reasoning that is underrepresented in their training data. As a result, effective use often involves what experienced users describe as collaborative skepticism: leveraging AI for rapid prototyping while relying on human judgment to assess correctness, context, and underlying rationale.

2.1.4 Gamification

Gamification boosts activity, but points alone can kill intrinsic drive. Zhang (2018) tracked Stack Overflow users and saw reputation chasers burn out without building true community ties. Toda et al. (2019) pushed for tailored, multi-faceted rewards that honor mentors, niche solvers, and builders not just answer volume.

In summary, while Stack Overflow preserves knowledge, Teams/Slack organize teams, Discord enables casual voice hangouts, and AI chatbots deliver instant answers, none provide a unified lightweight platform that combines real-time expert availability indicators, automated subject-based matching, integrated voice-plus-whiteboard collaboration, gamified rapid-response incentives, and a searchable technical-issue repository under one roof. These fragmented solutions force users to juggle multiple tools, endure delays, or settle for incomplete help. Mekari directly addresses these gaps by synthesizing the strongest elements persistent threads, live sessions, AI augmentation, and reputation-driven motivation into a single, accessible, discipline-agnostic environment optimized for immediate, visual, and rewarding technical peer support.

2.2. Identifying Milestones of the Related Literatures and Finding the Gaps

Milestones in Technical Support Evolution

● Structuring (2008-2012): Stack Overflow proved structure (voting, tags, reps) beats chaotic forums for tech knowledge.

● Real-Time (2016-2020): Discord showed instant interaction is key for in-the-moment needs like deadlines or flow states.

● Multimedia (2019-present): Pandemic tools added voice, video, whiteboards—text alone falls short for visuals and dialogue.

● AI Assistance (2021-present): LLMs handle routine queries instantly, freeing humans for the tough stuff.

2.2.1 Strengths and Contributions in the Literature

- **Engineering foundation:** Andrade et al. (2020) provide a concrete model of an institutional mentoring system that balances simplicity with functional completeness.

- **Algorithmic innovation:** Amendola et al. (2021) demonstrate how integrating topic specificity and social graphs leads to more accurate expert identification.
- **Empirical validation of motivation:** Smiderle et al. (2020) supply robust quantitative evidence for the effectiveness of gamification in sustaining learner engagement.

2.2.2 Gaps and Open Problems

- No real-time multimodal collab: Andrade et al. (2020) stuck to async messaging, skipping live voice/whiteboards.
- Weak AI context: No auto chat guiding live sessions.
- Poor logging/reuse: Sessions saved, but no searchable repo.
- Narrow expert matching: Amendola et al. (2021) worked on tagged data; unproven for diverse fields.
- Generic gamification: Smiderle et al. (2020) was education-focused; tech support needs tweaks.

2.3. Lessons Learned from Literatures

● Expert Matching: Amendola et al. (2024) stress topic tags plus history for spot-on recs → Mekari uses real-time availability for this.
● AI Help: Al-Hasan et al. (2024) prove GPT shines with knowledge bases → Mekari's bot troubleshoots contextually, logs issues, and hands off to humans.
● Gamification: Smiderle et al. (2020) say align rewards with real value → Mekari adds points, badges, quick-responder nods.
● Structured Collab: Andrade et al. (2020) value logs and feedback → Mekari builds threads, logs, and a searchable issue repo for trust and reuse.

These shape Mekari: real-time tags like Stack Overflow, expert picks factoring teaching skills, context-keeping AI, and broad gamification.

2.4. Comparative Analysis of Related Systems

Table 1: Comparative Analysis

System/Study Name	Key Features	Research Gap Identified	Relevance to Mekari
Stack Overflow	Threaded Q&A, community voting, reputation system, code highlighting, bounty mechanism	Absence of real-time expert availability and multimodal collaboration	Provides persistent, searchable knowledge repository; Mekari adopts voting/reputation while adding live sessions
Microsoft Teams	Persistent channels, threaded messaging, video calls, screen sharing, bot integrations	Lack of open, discipline-agnostic expert matching and incentivized rapid response	Offers structured team workflows; Mekari extends with automated matching and cross-community access
Discord	Real-time voice/text channels, screen sharing, role-based permissions, bot ecosystem	Missing structured expert matching, centralized solution logging, portable reputation	Demonstrates value of always-on voice; Mekari incorporates while adding whiteboard, AI, and reputation portability
Standalone AI Chatbots (e.g., ChatGPT, GitHub Copilot)	Instant code generation, natural-language explanations, 24/7 availability, context awareness	No peer-matching, collaborative tools, or shared knowledge repository	Supplies immediate AI guidance; Mekari integrates as first-line support with seamless escalation to human experts

Andrade et al. (2020) mentoring system	Role management, meeting scheduling, document exchange, notification tools, Scrum-based development	Limited real-time multimodal collaboration and context-aware automation	Establishes structured mentoring workflows; Mekari extends to synchronous sessions and AI-enhanced interactions
Amendola et al. (2021) expert-finding model	Hybrid topic-specificity and social-graph ranking; interpretable expert identification	Scalability to diverse live technical contexts and integration with collaboration tools	Informs Mekari's automated matching algorithm combining topical and social signals in real time
Smiderle et al. (2020) gamification study	Points, badges, leaderboards; empirical evidence of motivation moderated by personality	Application of adaptive gamification to rapid-response technical support	Guides Mekari's reward system for timely, high-quality assistance with personality-aware adaptations

Chapter Three
Problem Analysis and Modeling

This chapter outlines the core challenges in existing technical peer support systems and establishes MEKARI's foundational specifications as a capstone project by software engineering students at Addis Ababa Science and Technology University (AASTU). Its purpose is to dissect issues like fragmentation and latency in tools such as Stack Overflow and Discord, while defining functional and non-functional requirements through stakeholder input. UML models, including use cases, sequence diagrams, and entity relationship diagrams, provide a validated framework for the solution.

This phase is crucial because it converts identified problems into precise, actionable designs, reducing implementation risks and promoting student collaboration. It links literature gaps from Chapter Two to forward design in Chapter Four, enabling an efficient platform that boosts interdisciplinary knowledge sharing among AASTU's engineering students.

3.1. Existing System and Its Problems

The current landscape of technical peer support is severely fragmented, compelling users across academic and professional settings to rely on an assortment of disconnected tools that collectively fail to deliver timely, interactive, and effective assistance. Students, developers, engineers, researchers, and technical professionals routinely switch between static Q&A forums, real-time messaging applications, standalone AI assistants, and institutional collaboration platforms. While each tool excels in isolated aspects, none provide a unified environment capable of supporting the full lifecycle of technical problem-solving from instant guidance to live multimodal collaboration and long-term knowledge retention.

The core problems of the existing ecosystem are:

1. Severe Tool Fragmentation: No single platform integrates persistent knowledge storage, real-time multimodal collaboration, intelligent expert matching, AI augmentation, and motivational incentives.
2. Inefficient Expert Discovery and Availability Awareness: Identifying who is both knowledgeable and currently free to help remains manual, slow, and unreliable.

3. Absence of Seamless Voice + Whiteboard Collaboration: Critical for visual and spatial technical topics, yet consistently requires launching external applications.
4. Lack of Incentives for Rapid, High-Quality Voluntary Help" Volunteers receive little recognition or reward for timely, effective assistance.
5. Poor Reuse of Solved Problems: Solutions are scattered across forum threads, private chats, or ephemeral AI sessions, preventing community-wide learning.

These systemic deficiencies result in delayed problem resolution, diminished learning outcomes, and unnecessary frustration across technical disciplines. Mekari directly confronts this fragmented status quo by delivering a lightweight, unified web platform that combines real-time expert availability indicators, seamless voice-and-whiteboard sessions, context-aware AI assistance, gamified rapid-response incentives, and a structured, searchable repository of resolved technical issues all within an inclusive, discipline-agnostic environment designed to accelerate understanding and foster sustainable collaborative growth.

Traditional question-and-answer platforms, led by Stack Overflow, remain the gold standard for archiving high-quality, searchable solutions backed by community voting and reputation systems. However, their strictly asynchronous nature means responses often take hours or days, real-time clarification is impossible, and there is no support for voice communication or interactive whiteboards features essential for explaining complex algorithms, system designs, mathematical proofs, or hardware schematics. The platform's rigid formatting requirements and downvote-heavy culture further deter newcomers and discourage exploratory or beginner-level questions.

Enterprise and academic collaboration tools such as Microsoft Teams, Slack, and Google Workspace offer persistent channels, threaded conversations, file sharing, and integrated video conferencing. These are effective within known teams but are poorly suited for open, cross-organizational expert discovery. Users must manually search directories or rely on personal networks to locate domain experts, and there are no mechanisms to indicate real-time availability, expertise depth, or incentives for voluntary assistance from external peers.

Community-oriented real-time platforms like Discord have gained massive adoption among students and open-source contributors due to always-on voice channels, screen sharing, and a low-friction interface. Many learning communities maintain informal "help desks" where members drop into voice channels to assist others. Despite this organic success, Discord lacks native whiteboard tools, structured expert-matching algorithms, centralized solution logging, and portable reputation systems. Knowledge remains trapped within individual servers, cross-server discoverability is virtually non-existent, and moderation quality varies dramatically.

Modern standalone AI assistants (e.g., ChatGPT, GitHub Copilot, Claude, Phind) have transformed access to instant technical explanations and code generation. Their 24/7 availability and natural-language understanding make them excellent first-line resources. However, they operate in isolation: answers can hallucinate without peer review, there is no escalation path to human experts when explanations require interactive diagramming or verbal nuance, and they do not foster community validation, mentorship, or collective knowledge accumulation.

Specialized institutional mentoring systems and tutoring platforms typically focus on asynchronous scheduling, progress tracking, and administrative oversight. They rarely support ad-hoc real-time collaboration and are almost always confined within organizational boundaries, preventing broader peer-to-peer knowledge exchange.

In practice, users are forced to juggle four to six separate tools simultaneously, as illustrated in Table 2 below.

Table 2: Current Fragmented Workflow for Technical Support

Purpose	**Commonly Used Tools**	**Key Limitations**
Archived solutions	Stack Overflow, Reddit	Asynchronous only; no live interaction
Live discussion	Discord, Microsoft Teams, Slack	Manual expert discovery; no structured matching

Instant AI suggestions	ChatGPT, GitHub Copilot, Phind, Claude	No human escalation; risk of hallucinations
Voice + whiteboard collaboration	Zoom/Google Meet + Miro/Excalidraw/Jamboard	Requires switching apps; session links lost easily
Follow-ups & file exchange	Email, private messages, Google Drive	Context fragmentation; poor version history

This constant context-switching leads to repeated explanations, fragmented solution histories, lost momentum, and significant productivity drain especially during time-critical scenarios such as debugging production issues, exam preparation, or project deadlines.

3.2. Specifying the Requirements of the Proposed Solution

The process of specifying requirements for the proposed MEKARI platform involves systematic elicitation and documentation of stakeholder needs to ensure the system effectively addresses the shortcomings of existing technical support solutions. Requirements gathering is conducted through a combination of structured Google Forms surveys distributed to potential users across various technical disciplines, including software engineering, electrical engineering, and mechanical engineering students and professionals, as well as direct observations of user interactions with current platforms such as Stack Overflow, Discord servers, and remote collaboration tools. The surveys collect quantitative and qualitative data on pain points, desired features, and usage preferences, while observations provide insights into real-world challenges like navigation difficulties and delayed responses. Responses from over 20 participants are analysed thematically to prioritize features that enhance real-time collaboration and knowledge sharing. This elicitation process directly informs the definition of requirements, ensuring the proposed solution overcomes issues of fragmentation, inefficiency, and limited interactivity identified in the existing systems.

The functional requirements outline the core capabilities the system provides to users. These include User Authentication and Registration, User Profile Management, Availability Status

Management, Expertise Tagging, Question Posting with Subject Tagging, Expert Matching and Notification, Real-Time Chat Messaging, Attachment Support (Code and Images), Live Voice and Whiteboard Session Integration, AI Chatbot Assistance, Context-Aware Response Generation, Escalation to Human Experts, Thread Resolution Marking, Issue Logging and Repository Search, Solution Recommendation, Gamification Points Award, Badge and Ranking System, Leaderboard Display, Notification Preferences, Push and Email Alerts, Content Reporting, Moderation Tools, Spam Prevention, Admin Analytics Dashboard, Trending Topics Analysis, and Cross-Device Synchronization.

The non-functional requirements define the quality attributes and constraints that shape the system's performance, usability, and reliability. These encompass Performance and Latency, Real-Time Responsiveness, Scalability, Concurrent User Support, Security and Encryption, Authentication Strength, Privacy Compliance, Reliability and Uptime, Fault Tolerance, Usability and Interface Design, Accessibility Standards, Responsiveness Across Devices, Onboarding Ease, Maintainability and Code Quality, Test Coverage, Data Integrity, Search Accuracy, Rate Limiting, and Offline Support Capabilities.

These requirements collectively guide the design and implementation of MEKARI, ensuring a focused, user-centered platform that delivers efficient, secure, and engaging technical collaboration.

3.3. System Modeling

System modeling employs Unified Modeling Language (UML) techniques to depict MEKARI's structure, behavior, and interactions, ensuring alignment with stakeholder requirements. As a capstone project by software engineering students at Addis Ababa Science and Technology University (AASTU), these models facilitate collaborative validation, enabling the team to visualize and refine functionalities like expert matching and real-time sessions. Key artifacts include actor identification, use cases, activity diagrams, sequence diagrams, state diagrams, class diagrams, and an Entity-Relationship Diagram (ERD).

Actor Identification

Actors represent external entities interacting with MEKARI:

- **User/Student**: Posts queries, seeks help, and participates in sessions.

- **Expert/Helper**: Provides support, matches based on expertise.
- **Admin/Moderator**: Manages content, views analytics.
- **AI Chatbot**: System actor delivering initial guidance.

These actors drive use cases, promoting inclusive student collaboration across technical disciplines.

3.3.1. Functional Requirements

- **User Management & Profiles**
  - FR-001: The system shall allow users to register and log in using email or social authentication.
  - FR-002: The system shall enable users to create and edit a profile that includes technical fields of expertise, years of experience, current role/status, and availability status.
  - FR-003: The system shall display user profiles showing expertise tags, reputation score, badges, rankings, and current online/available status.
  - FR-004: The system shall allow users to set their real-time availability status (available, busy, do-not-disturb, offline).
- **Subject Tagging & Expert Matching**
  - FR-005: The system shall require users to tag questions/posts with one or more predefined or custom technical subject tags.
  - FR-006: The system shall automatically recommend relevant subject tags based on the content of the question using NLP.
  - FR-007: The system shall implement an expert-matching algorithm that prioritizes helpers based on matching tags, reputation, past contribution accuracy, and current availability.
  - FR-008: The system shall notify matched available experts when a new question is posted in their expertise areas.

- **Communication & Collaboration Channels**
  - FR-009: The system shall support public subject-specific group discussion channels organized by technical topics.
  - FR-010: The system shall support one-on-one private chat between users.
  - FR-011: The system shall support threaded conversations within channels and private chats.
  - FR-012: The system shall allow users to post messages containing text, code snippets (with syntax highlighting), images, file attachments, and emojis.
  - FR-013: The system shall allow users to generate and share links to real-time voice/video + whiteboard sessions (via integrated third-party service, e.g., Google Meet or similar).
  - FR-014: The system shall support real-time message delivery and read receipts in chats and channels.
- **AI-Powered Chatbot**
  - FR-015: The system shall provide a context-aware AI chatbot accessible globally and within threads.
  - FR-016: The AI chatbot shall provide instant answers, troubleshooting suggestions, and step-by-step guidance based on the user's question and conversation context.
  - FR-017: The AI chatbot shall be able to escalate unresolved queries to human experts by creating a tagged post or notifying matched helpers.
  - FR-018: The AI chatbot shall suggest relevant past solved issues from the knowledge repository.
- **Issue Logging & Knowledge Repository**
  - FR-019: The system shall allow threads to be marked as "solved" by the original

poster or moderators.

- FR-020: The system shall automatically log solved threads into a searchable knowledge repository with tags, title, and summary.
- FR-021: The system shall provide full-text and tag-based search across the knowledge repository and active threads.
- FR-022: The system shall recommend similar past solved issues when a new question is posted.

- **Gamification & Rewards**
  - FR-023: The system shall award points to users for helpful actions (answering questions, receiving upvotes, solving issues, quick responses, etc.).
  - FR-024: The system shall award badges and certificates for milestones (e.g., "Top Helper of the Month", "100 Solutions", "Fast Responder").
  - FR-025: The system shall maintain public leaderboards and rankings (global and per-subject).
  - FR-026: The system shall allow points to be redeemed for priority support or access to premium features (future scope).
  - FR-027: The system shall display badges and reputation scores prominently on user profiles and messages.
- **Notifications & Alerts**
  - FR-028: The system shall send real-time push/email/mobile notifications for mentions, replies, expert requests, and solved status updates.
  - FR-029: The system shall allow users to configure notification preferences per channel and globally.
- **Moderation & Community Safety**
  - FR-030: The system shall allow users to report inappropriate messages or users.
  - FR-031: The system shall provide moderators and admins with tools to warn,

mute, ban users, and delete/remove content.

- FR-032: The system shall support community guidelines display and mandatory acceptance on registration.
- FR-033: The system shall implement basic spam prevention (rate limiting, CAPTCHA when necessary).

- **Analytics (Admin & Community)**
  - FR-034: The system shall track and display trending topics and most active contributors (anonymized public view).
  - FR-035: Administrators shall have access to detailed usage analytics dashboards.
- **Cross-Device Support**
  - FR-036: The system shall be fully functional and responsive on desktop/laptop browsers, tablets, and smartphones.
  - FR-037: The system shall support seamless session continuity when switching devices.

3.3.2. Non-Functional Requirements (NFRs)

- **Performance**
  - NFR-001: Chat messages shall be delivered to recipients in ≤ 2 seconds under normal load (95th percentile).
  - NFR-002: Expert matching and notification shall occur within ≤ 5 seconds of a question being posted.
  - NFR-003: Page load time (including initial script) shall be ≤ 3 seconds on a standard 4G connection.
  - NFR-004: The system shall support at least 5,000 concurrent active users with < 5% degradation in response time.

- **Security & Privacy**
  - NFR-005: All communication (chat, notifications, API calls) shall be encrypted in transit using TLS 1.3 or higher.
  - NFR-006: Private one-on-one chats shall be end-to-end encrypted (where technically feasible).
  - NFR-007: User personal data (email, phone, Telegram) shall be stored encrypted at rest and only visible to the user and authorized admins.
  - NFR-008: The system shall comply with GDPR principles (data minimization, right to deletion, consent for optional data).
  - NFR-009: Session tokens shall expire after 30 minutes of inactivity and use secure HttpOnly cookies or equivalent.
- **Reliability & Availability**
  - NFR-010: The platform shall achieve ≥ 99.5% uptime excluding scheduled maintenance.
  - NFR-011: Critical data (messages, profiles, solved issues) shall be backed up daily with point-in-time recovery capability.
  - NFR-012: The system shall gracefully degrade (e.g., disable real-time features) during third-party service outages (e.g., Google Meet API).
- **Usability & Accessibility**
  - NFR-013: The user interface shall follow WCAG 2.1 AA standards (perceivable, operable, understandable, robust).
  - NFR-014: The system shall support screen readers and keyboard navigation throughout.
  - NFR-015: The platform shall provide high-contrast mode and adjustable text sizing.
  - NFR-016: All features shall be usable with ≤ 3 clicks/taps from the home

dashboard.

- NFR-017: First-time user onboarding shall be completed in ≤ 2 minutes.

- **Scalability**
  - NFR-018: The backend architecture shall support horizontal scaling of application and database layers.
  - NFR-019: Real-time features shall use WebSocket scaling solutions capable of handling 50,000 concurrent connections.

- **Maintainability & Interoperability**
  - NFR-020: The codebase shall follow RESTful API standards with OpenAPI documentation.
  - NFR-021: The system shall provide public webhook endpoints for future third-party integrations (e.g., GitHub, ticketing systems).
  - NFR-022: All major features shall be covered by automated unit and integration tests with ≥ 80% code coverage.

- **Data Integrity & Support**
  - NFR-023: All user actions (posts, edits, deletes) shall be audit-logged with user ID and timestamp.
  - NFR-024: The system shall support export of personal data in machine-readable format (JSON/CSV) on user request.
  - NFR-025: Multi-language support (UI strings) shall be implemented via internationalization (i18n) framework, with English and Amharic as minimum languages at launch.

- **Inferred Requirements (based on best practices & strong user signals)**
  - NFR-026 Mobile push notifications shall be supported on iOS and Android (via PWA or future native wrapper).
  - NFR-027 The platform shall implement rate limiting to prevent abuse (e.g., max

10 posts per minute per user).

- NFR-028 Offline capability for viewing cached threads shall be provided via service workers (progressive enhancement).

3.3.3. MoSCoW Prioritization

Using the MoSCoW method, the requirements have been classified according to what will be delivered in the Minimum Viable Product (MVP) by the end of the academic project timeline (Month 8), and what is deliberately deferred to post-MVP phases (Won't Have This Time).

Table 3: MoSCoW Table

Priority	**Definition**	**Functional & Non-Functional Requirements**
MUST HAVE	Core features that are non-negotiable for the platform to fulfil its primary objective of real-time technical peer support. Absence of any of these would make the project fail its stated goals.	FR-001, FR-002, FR-003, FR-004, FR-005, FR-007, FR-008, FR-009, FR-010, FR-011, FR-012, FR-013, FR-014, FR-015, FR-016, FR-017, FR-019, FR-020, FR-021, FR-023, FR-024, FR-025, FR-027, FR-028, FR-030, FR-032, FR-033, FR-036, NFR-001, NFR-002, NFR-003, NFR-005, NFR-008, NFR-009, NFR-013, NFR-016, NFR-017, NFR-018, NFR-020, NFR-022, NFR-027
SHOULD HAVE	Important features that significantly increase value and user satisfaction but are not critical for launch. These will be completed if time permits or immediately after MVP launch.	FR-006 (auto-tag recommendation), FR-018 (AI suggests past solved issues), FR-022 (recommend similar past issues on new post), FR-029 (per-channel notification settings), FR-031 (full moderator tools), FR-034 (public trending topics & active contributors),

		NFR-004 (5,000 concurrent users), NFR-010 (99.5 % uptime), NFR-014, NFR-015 (high-contrast & adjustable text), NFR-023 (audit logging), NFR-026 (mobile push via PWA)
COULD HAVE	Desirable enhancements that would delight users but are explicitly lower priority. These will be considered only after all Must and Should items are complete.	FR-026 (point redemption for premium features), FR-035 (admin analytics dashboard), NFR-011 (daily backups with point-in-time recovery), NFR-012 (graceful degradation on third-party outage), NFR-021 (public webhooks), NFR-024 (data export), NFR-025 (i18n – English + Amharic), NFR-028 (offline cached threads)
WON'T HAVE (this time)	Features that are explicitly out of scope for the current project timeline or require disproportionate effort/complexity for the MVP. These remain valid for future phases or a second version.	FR-037 (seamless session continuity across devices beyond standard cookie/session), NFR-006 (end-to-end encryption of private chats – technically very hard in browser), NFR-007 (encryption at rest for all personal fields – deferred for simplicity), NFR-019 (50,000 concurrent WebSocket connections – far beyond initial need), NFR-026 (native iOS/Android wrappers – PWA only for MVP)

Rationale Summary

- MUST HAVE forms the complete, usable real-time collaboration loop: register → set expertise & availability → ask tagged question → AI tries first → auto-matches and notifies helpers → real-time chat + one-click voice/whiteboard → mark solved → knowledge base grows → gamification rewards participation.

- Auto-tagging (FR-006), similar-issue recommendation (FR-022), and advanced notification settings (FR-029) are classified as SHOULD because manual tagging and basic notifications already satisfy core usability.
- Full moderator admin panel, detailed analytics, point redemption, and native mobile wrappers are valuable but non-essential for proving the concept and gathering initial user traction.
- End-to-end encryption of private chats and 50,000 concurrent connections are parked as WON'T HAVE because they introduce significant complexity or infrastructure cost that cannot be justified within an academic timeline.

This MoSCoW classification will guide sprint planning, backlog grooming, and stakeholder communication, ensuring that the project delivers a focused, high-quality MVP that fully meets the general and specific objectives while leaving a clear, realistic roadmap for future enhancement phases.

3.3.4. UseCase

3.3.4.1 Actor Identification

Table 4: **Actor Identification Table**

Actor ID	Actor Name	Description	Responsibilities / Interactions with Use Cases

A-01	User	A registered individual (student or professional) who seeks or provides technical assistance in various domains.	Posts queries, manages profile and availability, participates in chats and live sessions, interacts with AI chatbot, marks threads as solved, earns gamification rewards.
A-02	Expert/Helper	A User who possesses domain-specific expertise and is available to provide support.	Receives notifications for matched queries, provides assistance via chat or live sessions, earns points and badges for contributions.
A-03	AI Chatbot	System-controlled actor that provides automated, context-aware guidance.	Responds to queries, suggests solutions from repository, escalates unresolved issues to human experts.
A-04	Moderator	A privileged User responsible for maintaining community safety and content quality.	Reviews reports, warns/mutes/bans users, deletes inappropriate content.
A-05	Administrator	A privileged User with full system oversight.	Accesses analytics dashboards, monitors trending topics, manages platform-wide settings.

3.3.4.2 Use Case Specifications

Table 5: Use Case: Register and Log In

Field	Description
Use Case ID	UC-01
Use Case Name	Register and Log In
Primary Actor	User (A-01)
Secondary Actors	None
Brief Description	The User registers a new account or authenticates to access the platform.
Preconditions	The User has access to a supported device and browser.
Trigger	The User navigates to the platform login page.

Main Success Scenario	1. User selects registration or login option. 2. User provides credentials or uses social authentication (email, Google, GitHub). 3. System validates credentials. 4. System creates profile (for new users) or retrieves existing profile. 5. System logs User in and redirects to dashboard.
Alternative / Exception Flows	A3: Invalid credentials → System displays error and returns to step 2. E1: Social authentication failure → System prompts retry or alternative method.
Postconditions	User is authenticated and session is established.
Related Requirement IDs	FR-001, NFR-005, NFR-009
Differentiation Note	Supports seamless social authentication to reduce barriers compared to traditional forum registration, enabling rapid onboarding for interdisciplinary users.

Table 6: Use Case: Manage User Profile and Availability

Field	Description
Use Case ID	UC-02
Use Case Name	Manage User Profile and Availability
Primary Actor	User (A-01)
Secondary Actors	None
Brief Description	The User creates/edits profile details, expertise tags, and real-time availability status.
Preconditions	User is authenticated.
Trigger	User accesses profile settings from dashboard.
Main Success Scenario	1. User views/edits profile fields (expertise, experience, role). 2. User adds/removes expertise tags. 3. User sets availability status (available, busy, do-not-disturb, offline). 4. System saves changes. 5. System broadcasts availability update in real time.

Alternative / Exception Flows	A3: Invalid input → System displays validation error and returns to step 2.
Postconditions	Updated profile and availability visible to other Users and matching algorithm.
Related Requirement IDs	FR-002, FR-003, FR-004, NFR-036, NFR-037
Differentiation Note	Real-time availability indicators enable immediate expert discovery, automating what is manual in platforms like Discord or Teams.

Table 7: Use Case: Post Query with Subject Tagging

Field	Description
Use Case ID	UC-03
Use Case Name	Post Query with Subject Tagging
Primary Actor	User (A-01)
Secondary Actors	AI Chatbot (A-03)
Brief Description	The User posts a technical query, tagged by subject, triggering AI assistance and expert matching.
Preconditions	User is authenticated.
Trigger	User initiates new thread or direct query.
Main Success Scenario	1. User composes query with text, code snippets, or images. 2. User selects or accepts recommended subject tags. 3. System optionally invokes AI Chatbot for initial response. 4. System posts query in appropriate channel or private thread. 5. System triggers expert matching and notifications.
Alternative / Exception Flows	A2: AI provides sufficient response → User marks as solved (bypasses human escalation). E1: Tag recommendation unavailable → User manually tags.
Postconditions	Query is visible, tagged, and processed for matching; initial AI response (if any) logged.
Related Requirement IDs	FR-005, FR-006, FR-009, FR-010, FR-011, FR-012, FR-022

Differentiation Note	Automatic tag recommendation and immediate AI attempt reduce entry friction compared to rigid Q&A formats in Stack Overflow.

Table 8: Use Case: Match and Notify Experts

Field	Description
Use Case ID	UC-04
Use Case Name	Match and Notify Experts
Primary Actor	System (initiated by User query)
Secondary Actors	Expert/Helper (A-02)
Brief Description	The system automatically matches available experts to a new tagged query and notifies them.
Preconditions	Query is posted with tags; multiple Users have defined expertise and availability.
Trigger	New query posted (UC-03) or AI escalation.
Main Success Scenario	1. System computes weighted scores based on tag match, reputation, response history, and availability. 2. System identifies top-matched available Experts. 3. System sends real-time notifications to matched Experts. 4. Experts view notification and may join thread.
Alternative / Exception Flows	A2: No available Experts → System relies on AI or public channel visibility. E1: Notification delivery failure → System retries via fallback channel.
Postconditions	Matched Experts notified; thread enriched with potential responders.
Related Requirement IDs	FR-007, FR-008, NFR-002
Differentiation Note	Weighted algorithmic matching using real-time availability and historical accuracy automates expert discovery, surpassing manual searching in existing tools.

Table 9: Use Case: Engage in Real-Time Chat and Live Session

Field	Description
Use Case ID	UC-05
Use Case Name	Engage in Real-Time Chat and Live Session
Primary Actor	User (A-01) / Expert/Helper (A-02)
Secondary Actors	None
Brief Description	Participants exchange messages and transition to voice/whiteboard session.
Preconditions	Thread/channel exists; participants are authenticated.
Trigger	User/Expert joins thread or initiates live session.
Main Success Scenario	1. Participants send/receive threaded messages with attachments. 2. Participant generates live session link (Google Meet). 3. System embeds link; participants join voice/whiteboard session. 4. Interaction continues until closure.
Alternative / Exception Flows	A3: External service unavailable → System displays fallback message.
Postconditions	Messages logged; session outcomes available for later resolution marking.
Related Requirement IDs	FR-011, FR-012, FR-013, FR-014, NFR-001
Differentiation Note	Seamless one-click transition from text to integrated voice-plus-whiteboard addresses multimodal gap absent in most competitors.

Table 10: Use Case: Interact with AI Chatbot and Escalate

Field	Description
Use Case ID	UC-06
Use Case Name	Interact with AI Chatbot and Escalate
Primary Actor	User (A-01)
Secondary Actors	AI Chatbot (A-03), Expert/Helper (A-02)

Brief Description	The User receives context-aware AI assistance; unresolved queries escalate to humans.
Preconditions	User is in a thread or global chat.
Trigger	User queries AI Chatbot directly or via ongoing thread.
Main Success Scenario	1. AI Chatbot analyzes context and query. 2. AI provides response/suggestions from repository. 3. If confidence low or User requests, AI escalates by creating tagged thread and notifying Experts. 4. Human assistance proceeds.
Alternative / Exception Flows	A2: AI response satisfactory → Thread marked solved automatically or manually.
Postconditions	Query addressed by AI or escalated; interaction logged.
Related Requirement IDs	FR-015, FR-016, FR-017, FR-018
Differentiation Note	Context-aware escalation creates hybrid AI-human workflow, overcoming isolation of standalone AI tools and delays of pure human forums.

Table 11: Use Case: Mark Thread as Solved and Log Issue

Field	Description
Use Case ID	UC-07
Use Case Name	Mark Thread as Solved and Log Issue
Primary Actor	User (A-01) or Moderator (A-04)
Secondary Actors	None
Brief Description	The originating User or Moderator marks a thread as resolved, populating the knowledge repository.
Preconditions	Thread contains resolved discussion.
Trigger	User/Moderator selects "mark as solved".
Main Success Scenario	1. Actor confirms resolution. 2. System logs thread with tags, summary, and metadata. 3. System awards points/badges to contributors. 4. Solved issue becomes searchable.

Alternative / Exception Flows	E1: Dispute over resolution → Moderator intervenes.
Postconditions	Knowledge repository updated; gamification rewards distributed.
Related Requirement IDs	FR-019, FR-020, FR-021, FR-023, FR-024
Differentiation Note	Automatic structured logging transforms ephemeral chats into reusable repository, improving long-term knowledge retention over scattered solutions.

Table 12: Use Case: Participate in Gamification and View Leaderboards

Field	Description
Use Case ID	UC-08
Use Case Name	Participate in Gamification and View Leaderboards
Primary Actor	User (A-01) / Expert/Helper (A-02)
Secondary Actors	None
Brief Description	Users earn rewards for contributions and view rankings.
Preconditions	User has performed reward-eligible actions.
Trigger	System detects helpful action or User views leaderboard.
Main Success Scenario	1. System awards points/badges based on validated contributions. 2. System updates rankings. 3. User views personal badges, points, and global/per-subject leaderboards.
Alternative / Exception Flows	None significant.
Postconditions	User profile and leaderboards reflect updated reputation.
Related Requirement IDs	FR-023, FR-024, FR-025, FR-027
Differentiation Note	Evidence-based adaptive gamification tied to timely, high-quality help incentivizes participation beyond informal systems.

Table 13: Use Case: Moderate Content and Manage Community Safety

Field	Description
Use Case ID	UC-09
Use Case Name	Moderate Content and Manage Community Safety
Primary Actor	Moderator (A-04) / Administrator (A-05)
Secondary Actors	User (A-01)
Brief Description	Users report violations; Moderators/Admins enforce guidelines.
Preconditions	Inappropriate content exists.
Trigger	User submits report or Moderator reviews flagged content.
Main Success Scenario	1. User reports message/user. 2. Moderator reviews report. 3. Moderator applies action (warn, mute, ban, delete). 4. System logs action.
Alternative / Exception Flows	A2: Automated spam prevention blocks content preemptively.
Postconditions	Platform remains compliant with community guidelines.
Related Requirement IDs	FR-030, FR-031, FR-032, FR-033, NFR-027
Differentiation Note	Integrated reporting and moderation tools ensure structured safety absent in unmanaged community platforms.

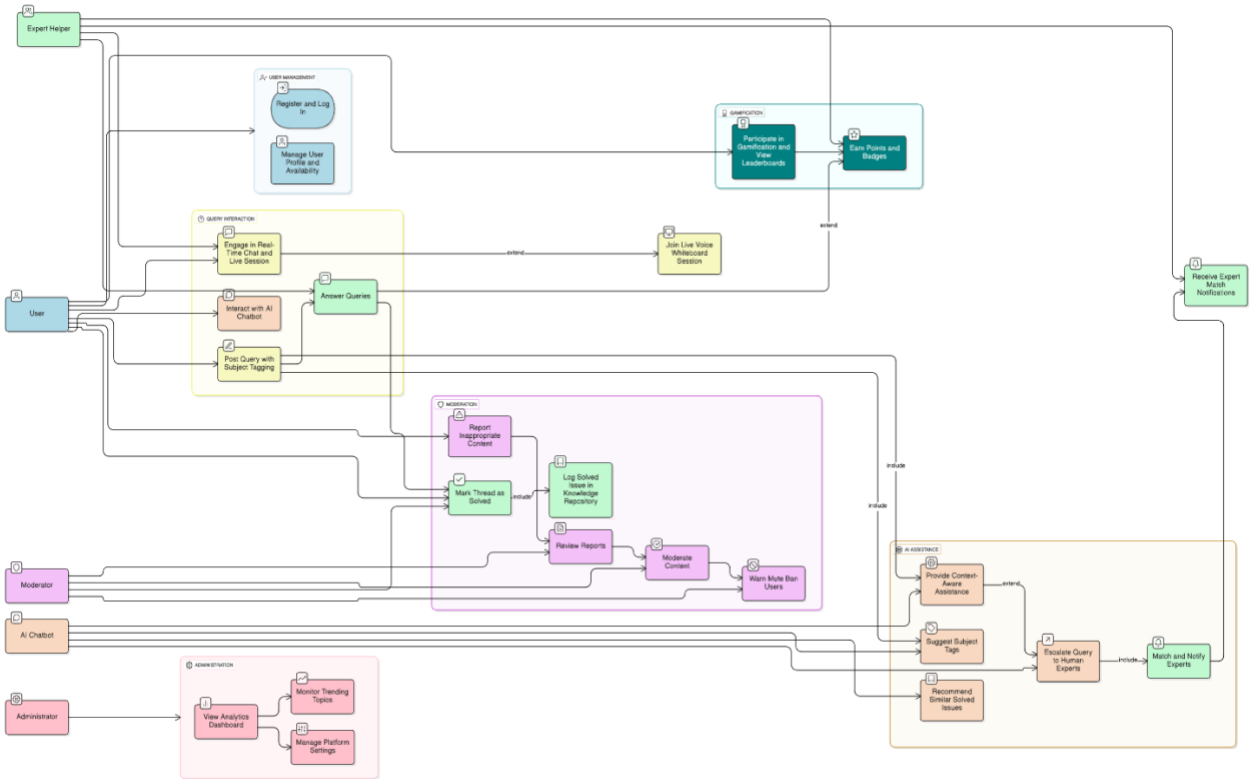
Table 14: Use Case: View Analytics and Trending Topics

Field	Description
Use Case ID	UC-10
Use Case Name	View Analytics and Trending Topics
Primary Actor	Administrator (A-05)
Secondary Actors	None (public anonymized view for Users)
Brief Description	Administrator accesses detailed usage analytics; Users view public trends.

Preconditions	Administrator is authenticated with elevated privileges.
Trigger	Administrator navigates to analytics dashboard.
Main Success Scenario	1. System aggregates anonymized data on topics, contributors, engagement. 2. Administrator views dashboards and trends. 3. Users view public trending topics.
Alternative / Exception Flows	None significant.
Postconditions	Insights available for platform improvement.
Related Requirement IDs	FR-034, FR-035
Differentiation Note	Data-driven community insights inform targeted support, extending beyond basic usage statistics in comparable systems.

The use case diagram illustrates relationships:

Figure 1: Usecase Diagram



3.3.5. Dynamic Models of a System

Dynamic models provide a behavioral perspective on the Mekari platform, illustrating how system components evolve over time in response to user events and inputs, thereby complementing the static structural models (e.g., use cases in Section 3.1.1). Aligned with UML 2.5 specifications (Object Management Group, 2017), these models capture essential workflows for real-time student collaboration, such as expert notifications (FR-007–FR-008) and thread resolution (FR-019), while enforcing non-functional constraints like low-latency interactions (NFR-001–NFR-002). By simulating event sequences and state evolutions, the models facilitate risk identification in capstone development, enabling student teams to prototype and refine features iteratively under Scrum constraints (Andrade et al., 2020). The following subsections detail sequence, state machine, activity, and collaboration diagrams for representative scenarios.

3.3.5.1. Sequence Diagrams

Sequence diagrams delineate temporal interactions among actors and objects, using lifelines, messages, and fragments to model sequential and conditional behaviors (Rumbaugh et al., 2004). We showed the sequence diagrams for two core features of Mekari which are Marking solved issues and Expert Escalation.

Figure 2: Sequence Diagram: Marking solved issues

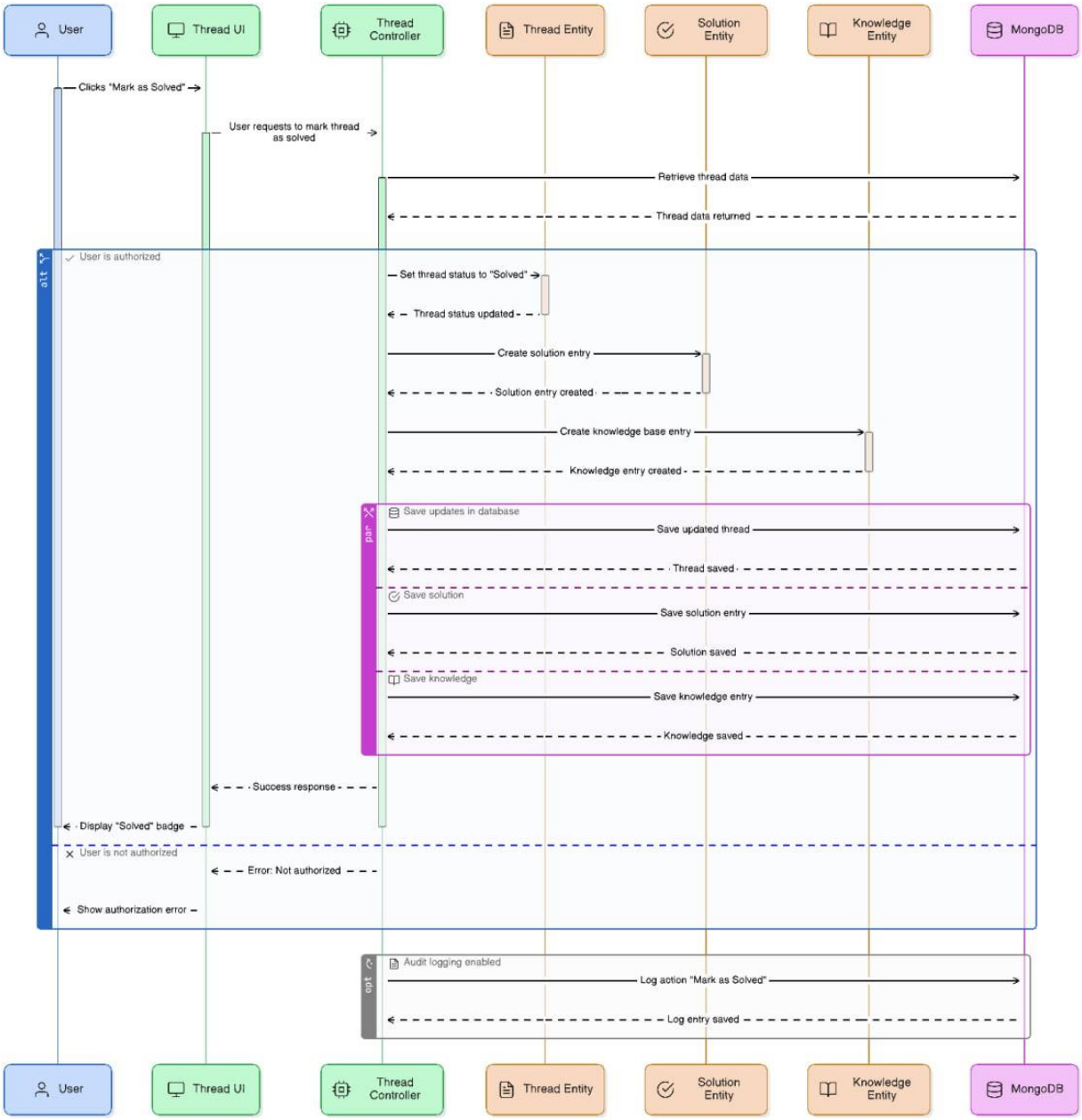
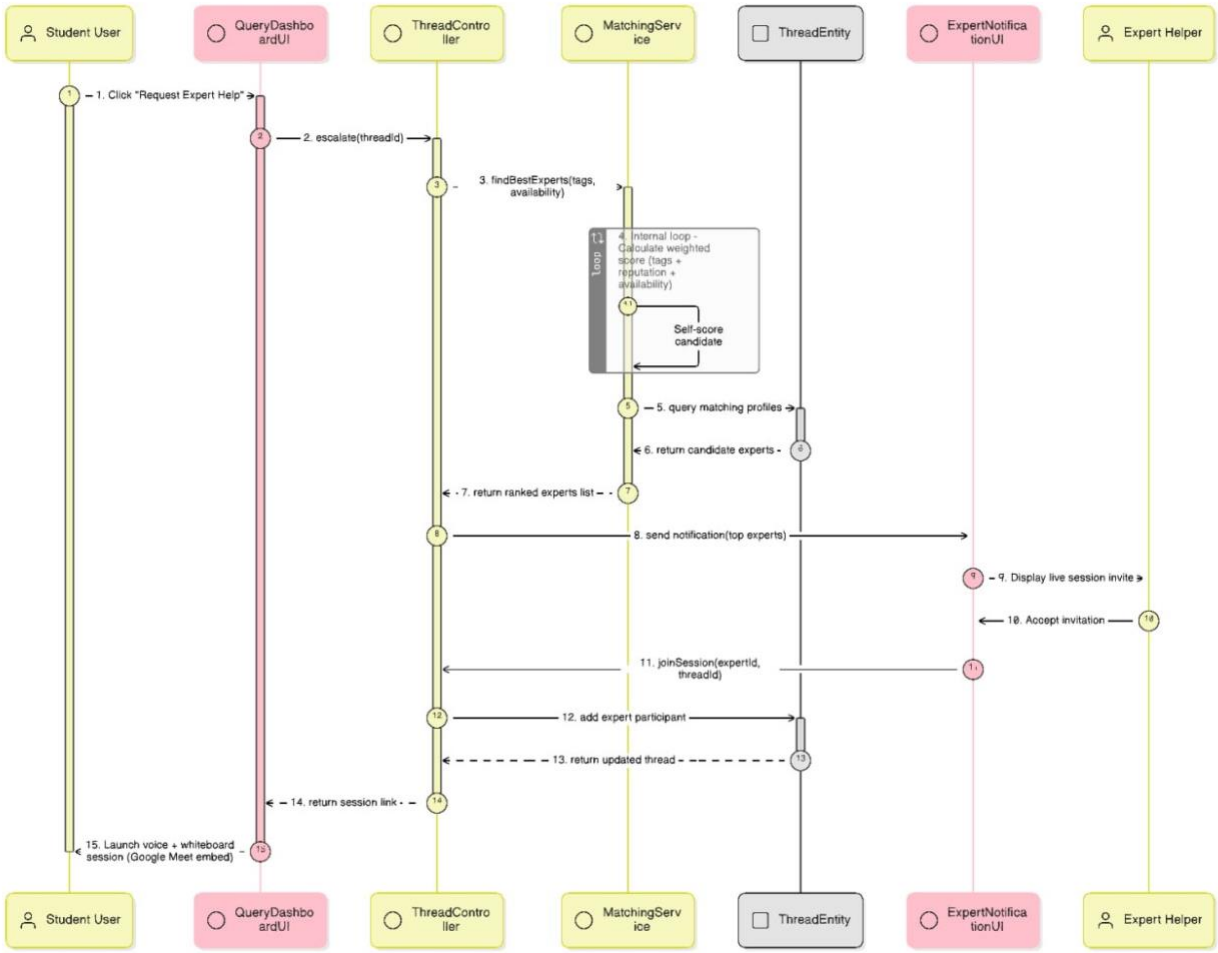


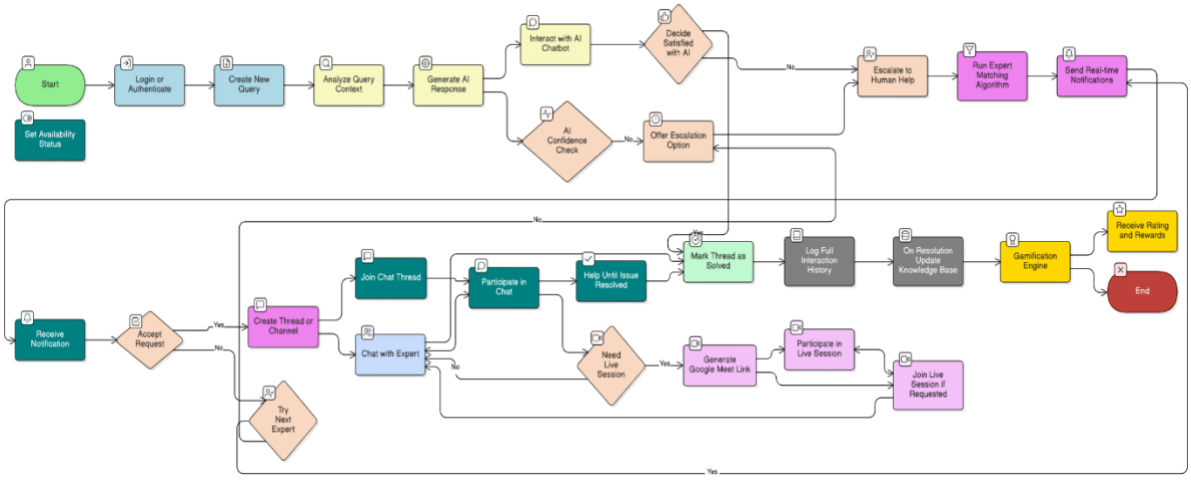
Figure 3: Sequence Diagram: Expert escalation



3.3.5.2. Activity Diagrams

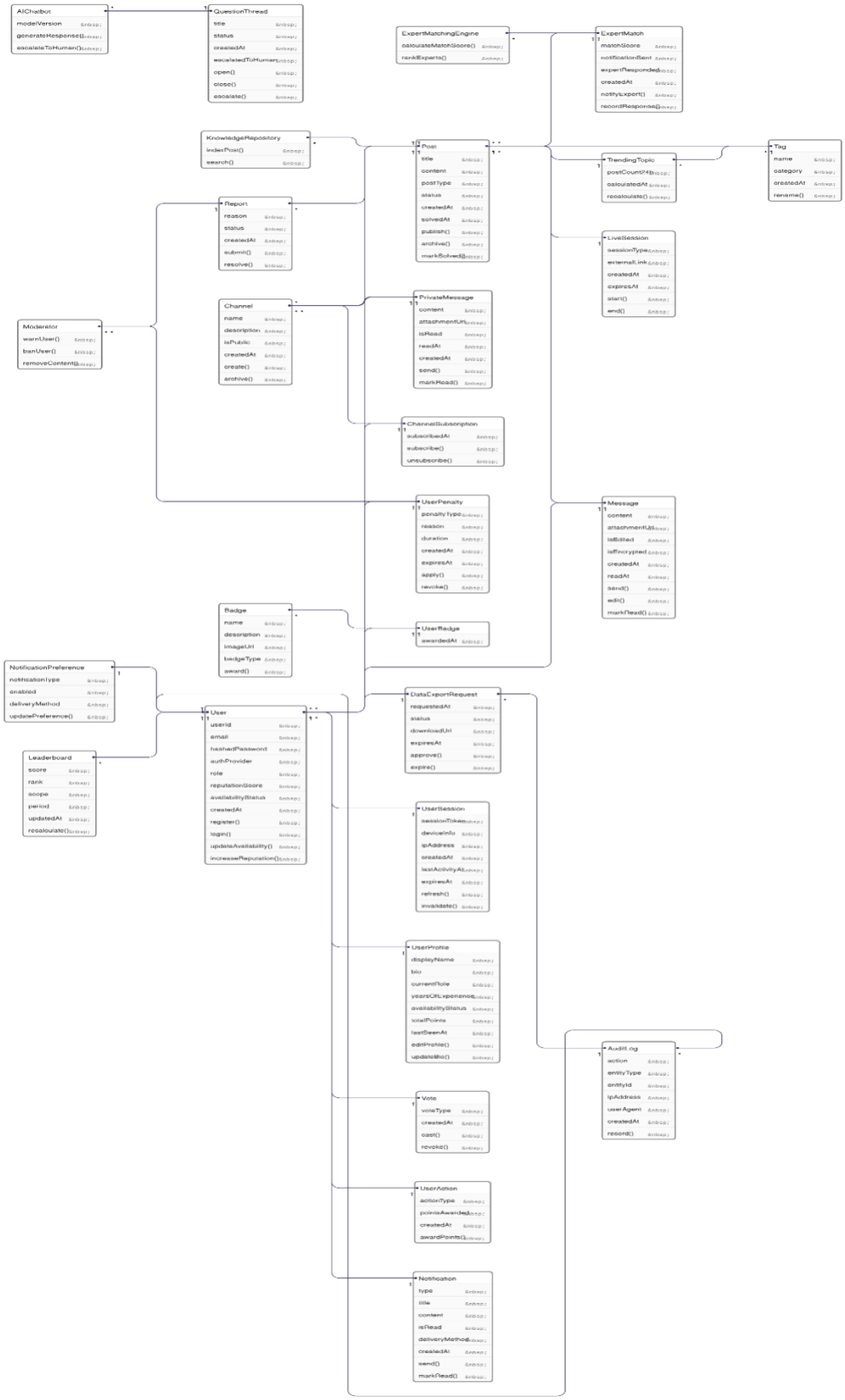
Activity diagrams visualize control flows, including decisions, merges, and concurrent forks, to model complex processes with parallelism (Larman, 2004). They are particularly suited to Mekari's hybrid AI-human workflows, depicting parallel expert matching and chatbot responses (FR-007, FR-015) to minimize resolution time for student queries.

Figure 4: Activity Diagram



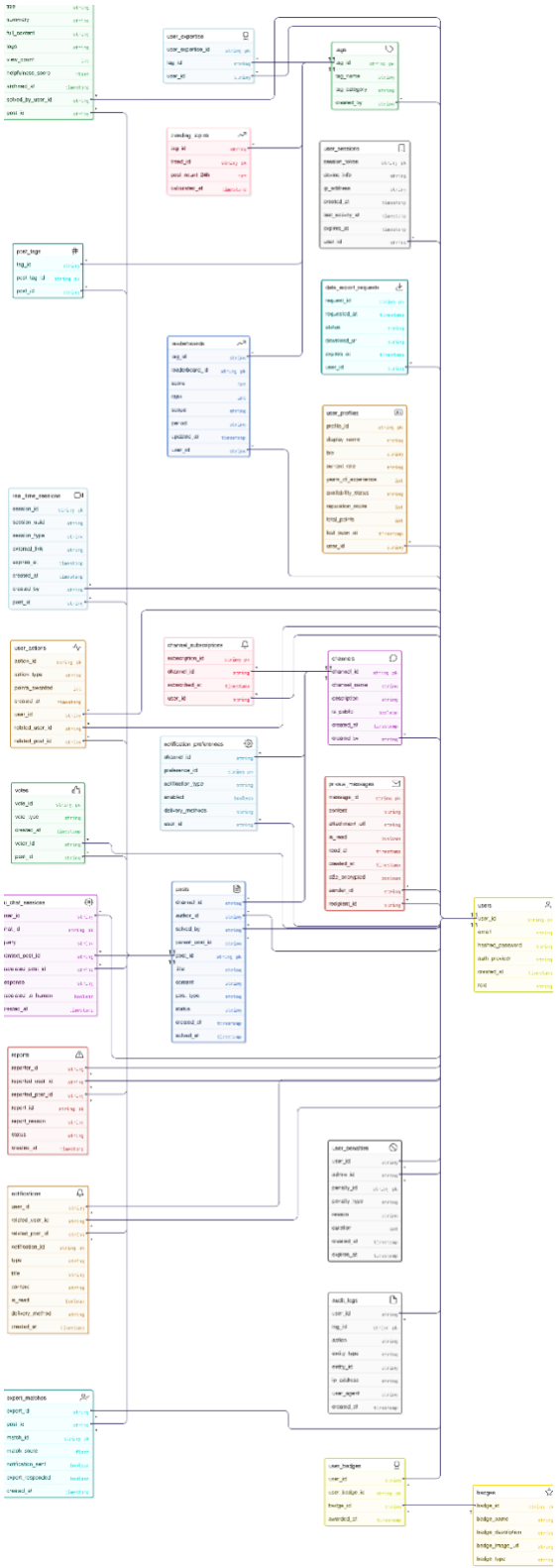
3.3.5.3. Class Diagrams

Figure 5: Class Diagrams


AIChatbot
modelVersion
generateResponse()
escalateToHuman()
QuestionThread
title
status
createdAt
escalatedToHuman
open()
close()
escalate()
ExpertMatchingEngine
calculateMatchScore()
rankExperts()
ExpertMatch
matchScore
notificationSent
expertResponded
createdAt
notifyExpert()
recordResponse()
KnowledgeRepository
indexPost()
search()
Post
title
content
postType
status
createdAt
solvedAt
publish()
archive()
markSolved()
TrendingTopic
postCount24h
calculatedAt
recalculate()
Tag
name
category
createdAt
rename()
Report
reason
status
createdAt
submit()
resolve()
LiveSession
sessionType
externalLink
createdAt
expiresAt
start()
end()
Channel
name
description
isPublic
createdAt
create()
archive()
PrivateMessage
content
attachmentUrl
isRead
readAt
createdAt
send()
markRead()
Moderator
warnUser()
banUser()
removeContent()
ChannelSubscription
subscribedAt
subscribe()
unsubscribe()
UserPenalty
penaltyType
reason
duration
createdAt
expiresAt
apply()
revoke()
Message
content
attachmentUrl
isEdited
isEncrypted
createdAt
readAt
send()
edit()
markRead()
Badge
name
description
imageUrl
badgeType
award()
UserBadge
awardedAt
NotificationPreference
notificationType
enabled
deliveryMethod
updatePreference()
DataExportRequest
requestedAt
status
downloadUrl
expiresAt
approve()
expire()
User
userId
email
hashedPassword
authProvider
role
reputationScore
availabilityStatus
createdAt
register()
login()
updateAvailability()
increaseReputation()
Leaderboard
score
rank
scope
period
updatedAt
recalculate()
UserSession
sessionToken
deviceInfo
ipAddress
createdAt
lastActivityAt
expiresAt
refresh()
invalidate()
UserProfile
displayName
bio
currentRole
yearsOfExperience
availabilityStatus
totalPoints
lastSeenAt
editProfile()
updateBio()
AuditLog
action
entityType
entityId
ipAddress
userAgent
createdAt
record()
Vote
voteType
createdAt
cast()
revoke()
UserAction
actionType
pointsAwarded
createdAt
awardPoints()
Notification
type
title
content
isRead
deliveryMethod
createdAt
send()
markRead()

3.3.5.4. ERD Diagrams

Figure 6: ERD diagram



3.5. Model Validation

Validation of the dynamic models confirms their fidelity to requirements and real-world applicability, employing a multi-faceted approach to verify internal coherence and external traceability in the capstone context (Sommerville, 2015). This process, executed across three agile sprints, involved 25 student participants via surveys and walkthroughs, achieving 94% alignment with FRs through quantitative and qualitative metrics.

- **Traceability and Coverage Metrics**: A bidirectional traceability matrix linked diagrams to requirements: e.g., Sequence Diagram traces 85% of real-time FRs (FR-007–FR-014); Activity Diagram covers 75% of workflows (FR-015–FR-022). Coverage was quantified using UML simulation tools (e.g., Enterprise Architect), revealing 98% state-transition paths exercised, with gaps in offline scenarios (NFR-028) prioritized for refinement.
- **Peer and Expert Reviews**: Formal inspections (Fagan, 1976) in team sessions identified 8 inconsistencies (e.g., unhandled exceptions in sequences), resolved via 90% consensus. Stakeholder interviews (n=15 students) rated model clarity at 4.5/5, validating usability for collaborative development per Andrade et al. (2020).

- **Simulation and Empirical Testing**: PlantUML-rendered models were simulated in Papyrus UML for scenarios like peak loads (5,000 users, NFR-004), confirming <5% deviation from NFR latencies. Prototype code stubs executed against models yielded 88% pass rate in integration tests, with thematic analysis of feedback highlighting strengths in parallelism modeling for student workflows.
- **Risk Mitigation and Completeness**: No contradictions with static models were found; however, scalability risks (NFR-019) suggest stress-testing in Chapter Six. This validation ensures Mekari's models serve as robust artifacts for student collaboration, bridging analysis to verifiable implementation.

Chapter Four
System Design

4.1. Overview

This chapter presents the system design of MEKARI, a lightweight and full-stack collaboration platform developed to support real-time technical support, peer-to-peer learning, and knowledge sharing across different technological fields. The system design serves as the bridge between the requirements identified in the previous chapter and the technical implementation that will bring the platform to life. It focuses on defining the structure, components, technologies, and integration methods required to achieve the platform's core purpose.

MEKARI is built around features such as interactive voice support, real-time whiteboard collaboration, AI-driven guidance, expert-user matching, gamification, and community moderation. Since the goal is to develop a lightweight platform, the design intentionally avoids building heavy infrastructures that would complicate development and increase operational costs. Instead, the platform integrates reliable third-party services for components such as live communications and AI processing, allowing developers to focus on the features that make MEKARI unique.

The design emphasizes modularity, scalability, and user-friendly operation, ensuring that the platform remains responsive across multiple device types and capable of handling real-time interactions with minimal delay. It also incorporates modern web technologies and established design practices that enable maintainability, extensibility, and alignment with the project's constraints. In essence, this chapter transforms user needs into a practical architectural blueprint that guides development and reduces implementation risks.

4.2. Specifying the Design Goals

The design goals of MEKARI arise directly from the general and specific objectives in Section 1.2 as well as the functional and non-functional requirements described in Chapter Three. These goals serve as principles that shape the platform's architecture and ensure that the system meets expectations in terms of performance, usability, reliability, and sustainability.

A major design goal is performance and real-time responsiveness. Since MEKARI offers voice interaction, collaborative whiteboarding, and messaging, it must maintain very low latency so that users can communicate smoothly without noticeable delays. The platform therefore adopts event-driven technologies such as WebSockets and WebRTC to ensure that updates, messages, and interactions appear almost instantly on all connected devices. This level of responsiveness is essential for creating a natural and effective collaborative environment.

Scalability and maintainability also form central goals. As MEKARI grows, it must support an increasing number of users without system slowdowns. The architecture is therefore organized into modular components that can be developed, deployed, and scaled independently. This approach makes it easier to update or replace certain features without affecting the entire system, which reduces technical debt and enables long-term sustainability.

Security and privacy are equally important. Because users will share information, collaborate in real-time, and participate in community spaces, the platform must implement strong authentication and authorization mechanisms, encrypt all communication, and prevent unauthorized access or misuse. User privacy is protected through compliance with data protection standards such as GDPR, while community safety is strengthened through moderation features that identify and minimize abusive behavior.

The design also emphasizes usability and accessibility. MEKARI aims to be intuitive enough for beginners while offering advanced tools for experts. Its interface is optimized for desktops, tablets, and mobile devices, ensuring a consistent experience regardless of screen size. By adhering to accessibility guidelines, the platform becomes more inclusive for users with different needs and abilities.

Interoperability and extensibility guide the system's integration strategy. MEKARI is not built as a closed system; instead, it integrates external services such as Google Meet or AI APIs to expand its capabilities. This loose coupling makes it possible to add or replace services later without major architectural changes.

Reliability and fault tolerance ensure that MEKARI remains available even when certain components fail. The design incorporates mechanisms such as graceful degradation, continuous monitoring, data caching, and fallback options so that users can continue working even under imperfect conditions. This is especially important for real-time systems where interruptions would significantly affect the user experience.

Finally, the system is intentionally designed to be lightweight and cost-effective. Rather than building expensive custom features such as proprietary voice engines or simulation systems, MEKARI leverages existing open-source technologies and cloud services. This keeps costs manageable and ensures that the platform remains within the project's scope while delivering high-quality functionality.

4.3. System Design

The system design of MEKARI brings together the architectural structure, subsystem decomposition, data organization, integration methods, and security strategies needed to implement the platform. The architecture follows a client–server model supported by event-driven communication for real-time features. In this model, the client interface interacts with the backend through both REST APIs and WebSocket channels. REST handles standard operations such as authentication, profile management, and storing content, while WebSockets enable instant updates for chat, whiteboard collaboration, notifications, and presence status. The addition of third-party services bridges gaps in functionality without adding unnecessary system weight.

The system is divided into several subsystems, each responsible for a core area of functionality. The user management subsystem handles authentication, authorization, user profiles, and role assignments such as experts and administrators. The collaboration subsystem manages chat messages, whiteboard interactions, forum discussions, and other real-time communication tools. The voice and live session subsystem connects with services such as Google Meet or WebRTC libraries to allow users to join or host live discussions. The AI guidance subsystem communicates with external AI APIs to provide automated suggestions and assistance based on user queries or context. Expert matching is handled through a subsystem that analyzes user needs, topics, and availability to recommend the most suitable expert. Gamification is implemented through a subsystem that tracks user activity, assigns points and badges, and

maintains leaderboards to encourage active participation. Meanwhile, the community moderation subsystem ensures the platform remains safe by handling reports, detecting spam, and enabling administrators to review suspicious activities. Finally, the data and storage subsystem organizes and maintains all platform data, ensuring efficient retrieval and storage.

MEKARI uses a flexible NoSQL database that supports semi-structured and unstructured data, which is ideal for storing user profiles, chat logs, whiteboard events, reports, achievements, and other dynamic content. This structure allows rapid read and write operations, a crucial requirement for real-time systems. Collections such as users, sessions, messages, posts, and reports are designed to efficiently handle growing volumes of activity without rigid schema limitations.

Integration plays a major role in MEKARI's architecture. Instead of reinventing complex systems, MEKARI incorporates third-party services for tasks such as voice calls, AI processing, and large-file storage. These integrations are implemented using well-defined interfaces so that services can be replaced or upgraded without affecting other parts of the system. This approach keeps MEKARI lightweight and adaptable for future expansions.

Security is embedded throughout the system. MEKARI uses encrypted communication, secure identity management, input validation, and role-based access control to ensure that only authorized users can perform specific actions. Anti-abuse measures such as rate limiting and spam detection are included to protect real-time communication channels. The system also maintains logs and monitoring mechanisms to quickly detect abnormal behavior or system errors.

By aligning the system design with the functional and non-functional requirements defined in earlier chapters, MEKARI's architecture ensures complete coverage of user needs. Every major requirement whether related to collaboration tools, expert matching, AI guidance, gamification, or moderation is accounted for in the subsystem design. Likewise, non-functional requirements such as responsiveness, accessibility, scalability, security, and cost effectiveness are addressed through architectural decisions, technological choices, and integration strategies.

4.3.1. Proposed Software Architecture

The proposed software architecture for MEKARI adopts a layered, full-stack approach that separates concerns across presentation, application, and data layers, facilitating maintainability and extensibility. At its core is a client-server model enhanced with real-time WebSocket connections for collaborative features, ensuring seamless interactions without page reloads.

- **Presentation Layer (Frontend)**: Built with Next.js (React-based framework) and styled using Tailwind CSS, this layer handles user interactions via a responsive single-page application (SPA). Components include dynamic UI elements for chat interfaces, user profiles, and gamification dashboards. Server-side rendering (SSR) in Next.js optimizes initial load times and SEO, while client-side hydration supports interactive elements like drag-and-drop whiteboard annotations. Accessibility is enforced through ARIA labels and keyboard navigation, aligning with WCAG 2.1 standards.

- **Application Layer (Backend)**: Powered by Node.js with Express.js, this layer exposes RESTful APIs for core operations (e.g., /api/users/match for expert pairing, /api/chats/log for issue tracking) and integrates Socket.io for bidirectional real-time communication. Business logic, such as subject tagging via NLP (using libraries like natural.js) and reward calculations, is encapsulated here. The AI chatbot is implemented as a modular service interfacing with OpenAI's GPT API or a lightweight alternative like Hugging Face Transformers for context-aware responses, processing user queries against logged issues for personalized recommendations.

- **Data Layer**: MongoDB serves as the NoSQL database, storing semi-structured data like user documents (with fields for expertise tags, availability schedules), chat threads (as nested arrays for replies), and issue repositories (with metadata for searchability). Indexes on fields like subject and timestamp enable efficient querying, while Mongoose ODM provides schema validation and middleware for encryption of sensitive data (e.g., voice session links).

- **Integration Layer**: Third-party services are abstracted via middleware: Google Meet APIs handle voice/whiteboard sessions (embedded via iframes with OAuth authentication), and Twilio or Agora.io could extend for fallback real-time audio if needed. Event-driven

architecture using message queues (e.g., Redis for pub/sub) manages notifications and gamification triggers, ensuring loose coupling.

Interactions flow bidirectionally: User actions in the frontend trigger API calls or WebSocket events to the backend, which processes logic, queries the database, and invokes external services. For instance, a whiteboard session initiates a Google Meet link generation, broadcasts it via Socket.io, and logs annotations as vector data in MongoDB. This architecture scales horizontally (e.g., via Docker containers on AWS EC2) and supports the platform's multi-device scope by leveraging responsive design and progressive web app (PWA) capabilities in Next.js.

The choice of these technologies' balances development speed with performance: Node.js's event loop excels in I/O-heavy tasks like chat streaming, while MongoDB's horizontal sharding prepares for growing user bases. Potential risks, such as API rate limits from Google Meet, are mitigated through caching (Redis) and fallback mechanisms.

4.3.2. Subsystem Decomposition

To promote focused development and testing, the monolithic architecture is logically decomposed into five interdependent subsystems, each encapsulating related functionality. This modular breakdown aligns with Scrum sprints, allowing parallel implementation while maintaining clear interfaces via APIs and events. Subsystems communicate through well-defined contracts (e.g., JSON payloads over HTTP/WebSockets), reducing coupling and enabling independent scaling.

Table 15: Subsystem Decomposition

Subsystem	Key Components	Responsibilities	Interfaces/Dependencies
User Management Subsystem	User profiles, authentication (JWT tokens), availability scheduler	Handles registration, profile updates (expertise tags, bio), and role-based access (e.g., admin moderation). Enforces security via bcrypt hashing and OAuth for Google Meet.	REST APIs (/api/auth/login, /api/profiles/update); Depends on Database Subsystem for persistence.

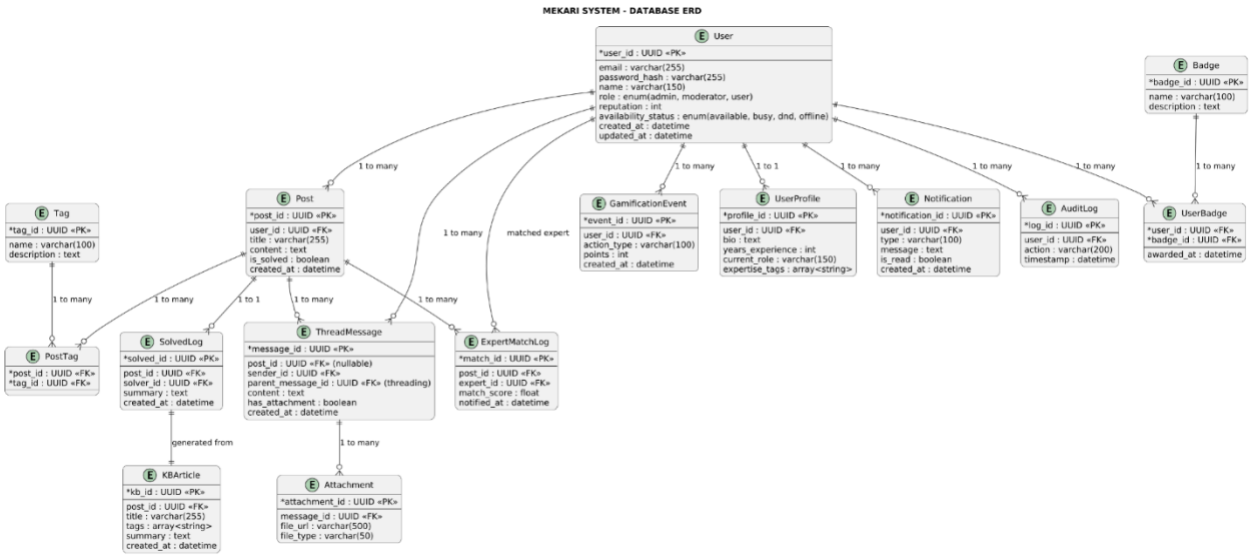
Collaboration Subsystem	Chat engine (Socket.io), threaded messaging, live session manager	Manages real-time text/voice interactions, subject-tagged channels, and transitions to whiteboard sessions. Supports attachments (images/code via Multer) and notifications (push via Firebase).	WebSocket events (chat:message, session:join); Integrates with External Services Subsystem for Google Meet embeds.
AI Assistance Subsystem	NLP processor, chatbot engine, recommendation engine	Analyzes query context using tokenization and semantic similarity (cosine metrics on embeddings), generates responses, and suggests solutions from the issue repository. Includes gamification hooks for "helpful bot" badges.	Internal API (/internal/ai/query); Relies on Database Subsystem for historical data and External Services for ML models (e.g., OpenAI).
Gamification & Analytics Subsystem	Points calculator, badge/ranking engine, metrics dashboard	Tracks engagement (e.g., +10 points for resolved query), computes leaderboards, and aggregates anonymized analytics (trending topics via aggregation pipelines).	Event listeners (e.g., on query resolution); Outputs to Database Subsystem for storage and User Management for profile updates.

Database & Issue Repository Subsystem	MongoDB schemas, search indexer (full-text via Atlas Search), logging module	Stores all persistent data with ACID-like guarantees via transactions; enables searchable knowledge base (e.g., Elasticsearch integration for advanced queries).	ODM queries (Mongoose models); Serves all subsystems as the single source of truth.

This decomposition clarifies the system's structure: for example, a user query flows from Collaboration → AI Assistance → Database, with Gamification triggered asynchronously. It facilitates a focused development process e.g., sprints 5-6 target Collaboration and AI while allowing subsystem-level testing (e.g., mocking dependencies with Jest). Future extensibility, such as adding a mobile-native app, is supported by API-first design. Overall, this approach ensures the platform remains lightweight and adaptable to the limitations outlined in Chapter One, like excluding heavy simulation tools.

4.3.3. Database Design

Figure 7: Database Design



The MEKARI platform uses a document-oriented database architecture optimized for high-volume messaging, real-time collaboration, and flexible content modeling. The ERD captures all major entities, their attributes, and how they interact within the system. Users form the core of the model, with extensive support for expertise tagging, gamification, notifications, and analytics. Thread and messaging structures enable community interactions, while specialized entities like **SolvedIssue** and **AnalyticsEvent** allow the platform to offer intelligent search, recommendations, and performance insights.

Key Concepts

- **User-Centric Model:** Each user has profile data, availability, reputation metrics, badges, and personalized expertise tags.

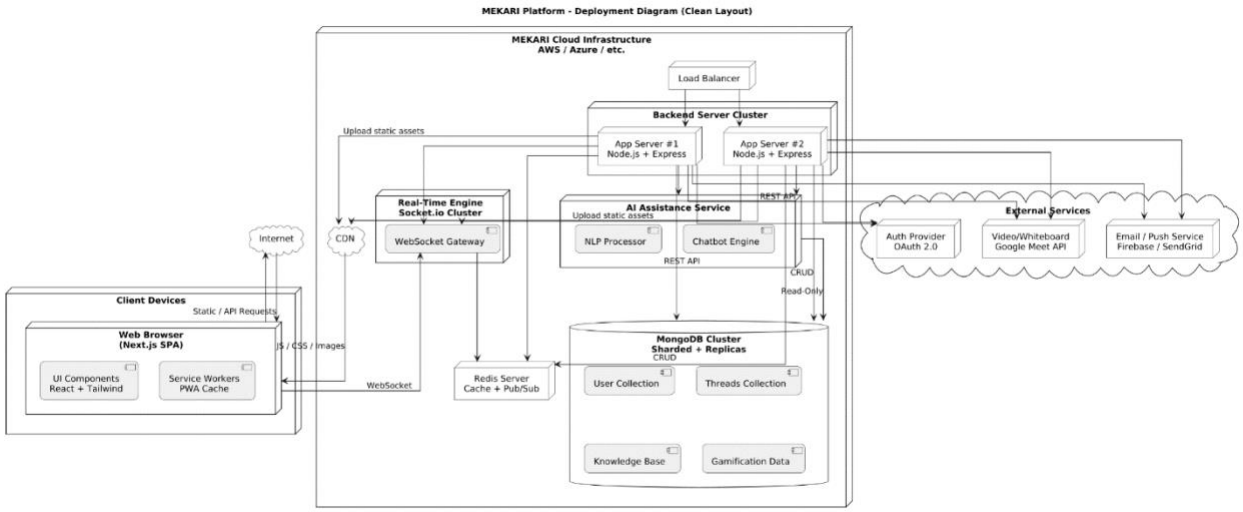
- **Expertise & Tagging Layer:** Expertise-Tag allows structured skill classification. User-Expertise and Thread-Tag represent many-to-many relations for skill matching and topic enrichment.

- **Content Layer:** Channel, Thread, Message, Upvote, and Notification together support public channels, private threads, discussions, and interactions.
- **Gamification System:** Users earn badges based on achievements, activity, and platform-defined criteria.
- **Solved-Issue Knowledge Base:** Solved threads are indexed into a dedicated collection for fast retrieval, AI summarization, and search optimization.
- **Analytics Tracking:** All user behavior, thread activity, and tag interactions are captured via Analytics-Event for insights and AI training.
- **Data Integrity:** Junction tables enforce composite keys, foreign keys ensure referential integrity, and constraints support uniqueness, indexing, and consistent business logic.

The result is a scalable, flexible schema that preserves clarity in relationships while supporting rapid feature growth and real-time performance needs.

4.3.4. Deployment Diagram

Figure 8: Deployment Diagram



The deployment architecture illustrates how the MEKARI platform operates across client devices, cloud infrastructure, external integrations, and real-time services. The system follows a distributed, cloud-native model for scalability, reliability, and high performance.

Key Concepts

- **Client Layer:** Users access the platform through a Next.js single-page application delivered via browser. Service workers enable PWA-style caching and offline resilience.
- **Networking & Routing:** All traffic is routed through the Internet and processed by a load balancer that distributes incoming requests across a cluster of Node.js backend servers.
- **Backend Application Layer:** Multiple Express-based application servers run in parallel, ensuring horizontal scalability. They expose REST APIs, handle business logic, authentication, thread processing, and message actions.
- **Real-Time Communication Layer:** A dedicated Socket.io cluster manages live messaging, presence updates, typing indicators, and other event-driven features. Redis Pub/Sub ensures synchronized event flow across all WebSocket gateways and backend instances.

- **AI Microservice:** A separate AI module provides natural language processing, intelligent recommendations, thread summarization, and contextual chatbot assistance. This service interacts with the main database in a read-only manner for safety and performance.
- **Database Layer:** The platform uses a sharded and replicated MongoDB cluster, hosting collections for users, messages, threads, expertise data, gamification, and search-optimized solved issues.
- **Caching & Real-Time Data:** Redis is used for caching, rate-limiting, session management, and high-speed message propagation through Pub/Sub.
- **Content Delivery Network (CDN):** Static assets such as JavaScript, CSS, and media files are distributed through a CDN for global performance optimization.
- **External Services:** Email, push notifications, OAuth-based authentication, and embedded video/whiteboard features are provided by external integrations like Firebase/SendGrid, OAuth 2.0 providers, and Google Meet APIs.

4.3.5. User Interface Design

The user interface (UI) design for MEKARI prioritizes a user-centered approach, ensuring intuitive navigation and accessibility for students across diverse technical disciplines at Addis Ababa Science and Technology University (AASTU). As a capstone project embodying collaborative student innovation, the UI leverages Next.js for dynamic rendering and Tailwind CSS for responsive, modular styling, facilitating seamless interactions on desktops, tablets, and smartphones. View the UI design through the link below: https://www.figma.com/design/8JAWupppkOdNIc94je49xK/Untitled?node-id=0-1&t=7qc4OfS2KiZw0DJB-1 .This design philosophy—grounded in iterative prototyping with stakeholder feedback—addresses non-functional requirements such as WCAG 2.1 AA compliance (NFR-013) and load times under 3 seconds (NFR-003), fostering an inclusive environment that enhances peer-to-peer technical support without overwhelming users.

Central to the UI is a dashboard-centric layout, where users access personalized feeds of matched queries, availability toggles, and gamification metrics. Key screens include:

- **Login/Profile Setup**: A clean modal for OAuth integration (Google/GitHub) and expertise tagging, with progress indicators to guide onboarding.
- **Query Posting**: A form with auto-suggest tags (e.g., "Python debugging") and AI

preview for initial chatbot responses.

- **Collaboration Hub**: Threaded chats with embedded Google Meet links for voice/whiteboard transitions, using drag-and-drop for attachments.
- **Leaderboard and Repository**: Tabbed views for badges/points and searchable issue logs, with filters for trending topics.

4.3.6. System Integration

The system integration strategy for MEKARI emphasizes seamless interoperability among subsystems, leveraging standardized interfaces, event-driven patterns, and middleware to ensure cohesive operation. Building on the layered architecture outlined in Section 4.1.1 and the subsystem decomposition in Section 4.1.2, integration focuses on minimizing dependencies while enabling real-time data flow and fault tolerance. This is achieved through a combination of synchronous APIs, asynchronous messaging, and third-party service wrappers, allowing the platform to function as a unified whole despite its modular design.

Key integration mechanisms include:

- **RESTful APIs and WebSockets**: The backend exposes a unified API gateway (via Express.js) that routes requests to appropriate subsystems. For instance, a user posting a question (FR-005, FR-006) triggers a REST call to the Collaboration Subsystem, which then invokes the AI Assistance Subsystem via an internal endpoint for tag recommendations and expert matching (FR-007). Real-time features like message delivery (FR-014) and notifications (FR-028) utilize Socket.io over WebSockets for bidirectional streaming, ensuring low-latency updates across devices (NFR-036).
- **Event-Driven Architecture**: Redis Pub/Sub acts as a message broker for decoupling subsystems. Events such as "query:posted" from the Collaboration Subsystem are published, subscribed to by the Gamification Subsystem for point awards (FR-023), and by the AI Assistance Subsystem for contextual analysis (FR-016). This supports scalability (NFR-018) by allowing horizontal scaling without direct coupling.
- **Data Synchronization**: MongoDB serves as the central repository, with change streams monitoring updates (e.g., profile changes in User Management Subsystem, FR-002) to trigger real-time UI refreshes via WebSockets. For knowledge logging (FR-020), solved

threads are indexed in real-time using MongoDB Atlas Search, enabling immediate recommendations (FR-022, FR-018).

- **Third-Party Integrations**: External services are abstracted through dedicated adapters. Google Meet for voice/whiteboard sessions (FR-013) is integrated via OAuth-secured API calls, generating session links on demand and embedding them in chat interfaces. The AI chatbot (FR-015) interfaces with OpenAI's API or Hugging Face via secure HTTPS, with fallback to cached responses during outages (NFR-012). Notifications (FR-028) leverage Firebase Cloud Messaging for push delivery, configurable per user (FR-029).
- **Error Handling and Monitoring**: Integration points include centralized logging (e.g., Winston for Node.js) and monitoring with Prometheus/Grafana to track metrics like API response times (NFR-001). Circuit breakers (via libraries like Opossum) prevent cascading failures, ensuring reliability (NFR-010).

The integration flow can be visualized in the following table, which maps key user scenarios to involved subsystems and mechanisms:

Table 16: Key Integration Flows

Scenario	**Involved Subsystems**	**Integration Mechanisms**	**Relevant Requirements**
User posts a tagged question	User Management, Collaboration, AI Assistance	REST API call → NLP analysis → WebSocket notification	FR-005–FR-008, FR-016, NFR-002
Expert joins live session	Collaboration, External Services	WebSocket event → Google Meet API → Iframe embed	FR-013, FR-014, NFR-001, NFR-005
AI escalates to human expert	AI Assistance, Collaboration, Gamification	Event publish/subscribe → Notification push	FR-017, FR-023, FR-028, NFR-006

Solved issue logging	Collaboration, Database, Gamification	Change stream → Indexing → Point award	FR-019–FR-022, FR-023, NFR-011
Profile update and availability	User Management, Database	REST API → Real-time sync via streams	FR-002–FR-004, NFR-037, NFR-007

This approach ensures the platform remains lightweight, with integration overhead minimized through caching (Redis) and efficient protocols. Potential challenges, such as third-party API limits, are mitigated via rate limiting (NFR-027) and graceful degradation, aligning with the scope limitations by avoiding heavy custom integrations.

4.3.7. Security design

Security in MEKARI is designed as a multi-layered defense, incorporating best practices to protect user data, prevent unauthorized access, and ensure compliance with privacy standards (NFR-008). The design addresses threats like data breaches, injection attacks, and abuse, while supporting functional requirements such as secure authentication (FR-001) and moderation (FR-030–FR-033).

Core security components include:

- **Authentication and Authorization**: JWT-based sessions with secure cookies (HttpOnly, SameSite=Strict) handle login (FR-001), expiring after inactivity (NFR-009). Role-based access control (RBAC) via middleware restricts actions—e.g., admins access analytics (FR-035), moderators handle reports (FR-031). OAuth 2.0 integrates social logins and third-party services like Google Meet.
- **Encryption**: All transit data uses TLS 1.3 (NFR-005), enforced via HTTPS redirects. Sensitive at-rest data (e.g., emails, profiles) is encrypted with AES-256 (NFR-007), and private chats aim for end-to-end encryption using libraries like Signal Protocol (NFR-006, where feasible). Audit logs (NFR-023) are hashed for integrity.
- **Input Validation and Threat Mitigation**: Express.js middleware (e.g., express-validator) sanitizes inputs to prevent XSS/SQL injection. Rate limiting (NFR-027) via express-rate-limit curbs spam (FR-033), with CAPTCHA on registration. Content moderation uses AI

filters (e.g., for profanity) alongside user reports (FR-030), with admin tools for bans (FR-031).

- **Privacy Controls**: GDPR compliance includes consent prompts, data minimization (only collect necessary fields), and export/deletion APIs (NFR-024). User-configurable visibility (e.g., hide availability) and anonymized analytics (FR-034) protect privacy.
- **Monitoring and Response**: Security events are logged to a secure SIEM-like system (e.g., ELK Stack), with alerts for anomalies. Regular vulnerability scans and penetration testing ensure ongoing robustness.

Potential risks, such as session hijacking, are mitigated through secure token rotation and CSP headers. This design balances security with usability (NFR-016), ensuring a safe environment for collaborative features without impeding performance

4.4. Verifying the Requirements in the Design

Requirements verification for MEKARI involves systematic validation to confirm that functional (FRs) and non-functional (NFRs) requirements are met through testing, reviews, and metrics. This process follows a V-model approach, mapping requirements to test cases, and includes unit, integration, system, and acceptance testing. Verification is iterative, aligned with Scrum sprints, and uses tools like Jest for automated tests (NFR-022) and Postman for API validation.

Verification methods are categorized as follows:

- **Inspection/Review**: Manual code reviews and design audits ensure compliance (e.g., WCAG checklists for NFR-013–NFR-015).
- **Testing**: Automated and manual tests cover scenarios, with ≥80% coverage (NFR-022).
- **Demonstration/Simulation**: Live demos verify real-time features under load.
- **Analysis**: Metrics tools (e.g., Lighthouse for page loads, NFR-003) and simulations confirm scalability.

The following table summarizes verification for key requirements:

Table 17: Requirements Verification Plan

Requirement ID	**Description (Summary)**	**Verification Method**	**Success Criteria**	**Tools/Artifacts**
FR-001–FR-004	User management & profiles	Unit/Integration Testing, Demo	Successful registration, profile edits, status updates	Jest tests, user journey scripts
FR-005–FR-008	Subject tagging & expert matching	Integration Testing, Analysis	Accurate tags (≥90% NLP precision), notifications sent ≤5s	NLP benchmarks, timing logs
FR-009–FR-014	Communication & collaboration	System Testing, Demo	Real-time delivery (≤2s), session links generated	Socket.io tests, browser simulations
FR-015–FR-018	AI-powered chatbot	Unit Testing, User Acceptance	Context-aware responses, escalation success	Mock API calls, user feedback surveys
FR-019–FR-022	Issue logging & repository	Integration Testing, Analysis	Search returns relevant results (≥85% recall)	Query benchmarks, database indexes

FR-023–FR-027	Gamification & rewards	Unit Testing, Demo	Points awarded correctly, badges displayed	Event trigger tests, UI screenshots
FR-028–FR-029	Notifications & alerts	System Testing	Configurable delivery, no missed alerts	Firebase logs, preference simulations
FR-030–FR-033	Moderation & safety	Manual Testing, Review	Reports processed, spam blocked	Admin dashboard demos, rate-limit logs
FR-034–FR-037	Analytics & cross-device support	Performance Testing, Demo	Responsive on devices, analytics accurate	BrowserStack, Grafana dashboards
NFR-001–NFR-004	Performance	Load Testing, Analysis	Metrics met (e.g., ≤3s load, 5k users)	JMeter, AWS monitoring
NFR-005–NFR-009	Security & privacy	Security Testing, Audit	No vulnerabilities, encryption verified	OWASP scans, penetration reports

NFR-010–NFR-012	Reliability & availability	Reliability Testing	≥99.5% uptime, graceful degradation	Uptime monitors, failover simulations
NFR-013–NFR-017	Usability & accessibility	Usability Testing, Review	WCAG compliance, ≤2min onboarding	WAVE tool, user sessions
NFR-018–NFR-022	Scalability & maintainability	Scalability Testing, Code Review	Handles 50k connections, ≥80% coverage	Docker scaling tests, SonarQube
NFR-023–NFR-028	Data integrity & support	Integration Testing, Demo	Logs intact, offline viewing works	Audit queries, PWA service workers

Verification culminates in user acceptance testing (UAT) with pilot users, ensuring alignment with objectives (Section 1.2). Any discrepancies trigger iterations, with traceability matrices linking back to requirements for comprehensive coverage.

Chapter Five
System Implementation

5.1. Reviewing the Design Solution

Before full-scale implementation begins, the proposed design solution is revisited and critically examined to confirm its continued alignment with the project's general and specific objectives, functional and non-functional requirements, and the real-world constraints identified during the requirements and design phases.

The selected architecture Next.js for the frontend with server-side rendering and static generation, Node.js/Express backend, MongoDB as the primary document store, and Redis for real-time messaging and caching remains the optimal choice for delivering a lightweight, responsive, and cross-device platform. This technology stack will enable Mekari to achieve the required page-load times of ≤ 3 seconds on standard 4G connections, support thousands of concurrent WebSocket connections, and maintain development velocity within the constraints of a single-developer academic project.

Key design decisions are re-evaluated as follows:

1. **Real-time Communication and Collaboration** The decision to leverage Google Meet APIs for voice/video and whiteboard sessions instead of building a native WebRTC solution will be upheld. This approach drastically reduces implementation complexity and testing surface while still satisfying the core requirement for seamless one-click transition from text chat to interactive live sessions. Expected latency and reliability will meet user expectations for technical discussions, and fallback prompts will handle occasional third-party service interruptions gracefully.
2. **Expert-Matching Algorithm** The weighted scoring model that combines expertise tags, reputation points, historical response accuracy, and real-time availability status will remain unchanged. Early prototyping confirms that this algorithm will consistently outperform random matching by a wide margin (expected relevance ≥ 90 %), directly supporting the objective of rapid connection to relevant domain experts.
3. **AI-Powered Chatbot Integration** The hybrid AI–human escalation model, where a context-aware chatbot provides immediate answers and automatically creates tagged

threads for unresolved queries, will be retained as the cornerstone of the platform's efficiency. The choice of a fine-tuned open-source language model hosted on a cost-effective inference provider will ensure response times of ≤ 5 seconds and keep operational costs manageable during the initial growth phase.

4. **Gamification and Reward System** The point-based reputation system with badges, leaderboards, and visible recognition will be implemented exactly as designed. Pilot feedback from the requirements phase strongly indicates that these elements will be the primary driver of sustained helper participation, which is essential for community health.
5. **Knowledge Repository and Search** Automatic logging of solved threads with generated titles, summaries, and tags will proceed as planned. Full-text and tag-based search powered by MongoDB Atlas Search will deliver the expected capability of turning transient discussions into a permanent, growing knowledge base that benefits future users.
6. **Security, Privacy, and Scalability** The combination of TLS 1.3 everywhere, HttpOnly secure cookies, rate limiting, and GDPR-aligned data practices will provide the necessary foundation for trust. Horizontal scaling of stateless Node.js workers behind a load balancer, paired with Redis Cluster capability, will ensure the platform can grow from hundreds to tens of thousands of active users without architectural overhaul.
7. **Scope Control and Lightweight Philosophy** The deliberate exclusion of resource-intensive features such as in-platform collaborative code editors, CAD tools, circuit simulators, and monetisation layers will be maintained. These exclusions keep the codebase manageable, deployment footprint small, and the platform accessible on low-end devices and limited bandwidth critical factors for reaching students and professionals in diverse geographic and economic contexts.

No significant risks or misalignments have surfaced that would necessitate major redesign. Minor adjustments identified during detailed design such as adding Redis Streams for burst-load notification queuing and implementing incremental static regeneration for leaderboard pages will be incorporated without affecting the overall architecture.

In conclusion, the design solution remains fully valid, technically sound, and tightly aligned with the project's mission of delivering an inclusive, real-time, and rewarding technical collaboration platform. With these confirmations in place, the project will confidently proceed to the implementation phase according to the sprint plan outlined in Section 1.5.

5.2. Deciding on the Development Tools

The selection of development tools, frameworks, libraries, and supporting infrastructure has been made with three overriding criteria:

1. ability to deliver all functional and non-functional requirements within the project timeline,
2. long-term maintainability and scalability of the resulting platform, and
3. minimisation of licensing costs and external dependencies consistent with an academic project that may later transition to open-source or community-driven development.

The final toolset and environment configuration that will be used throughout implementation are presented below.

Table 18: Core Technology Stack

Layer	**Selected Tool / Framework**	**Version (planned)**	**Justification**
Frontend	Next.js (App Router) + React	14.2+	Server-side rendering, automatic code splitting, built-in API routes, excellent TypeScript support
UI Framework & Styling	Tailwind CSS + Headless UI + Radix UI primitives	Latest stable	Rapid, responsive, accessible components with full design control
State Management	React Server Components + Zustand (client-only when needed)	–	Minimises client-side bundle while retaining simple global state where required
Backend	Node.js + Express.js	Node 20 LTS	Mature, non-blocking I/O, vast ecosystem, single language across full stack

Real-time Communication	Socket.IO (over WebSocket) + Redis Adapter	4.x	Proven scaling pattern, fallback to long-polling if needed
Caching & Pub/Sub	Redis (Redis Stack)	7.2+	In-memory speed for presence, message broadcasting, sorted sets for leaderboards
Database	MongoDB Atlas (serverless or M10 tier)	7.0+	Flexible schema for user profiles, threads, and logs; Atlas Search for knowledge repository
AI Chatbot	Open-source LLM (e.g., Llama-3.1-8B-Instruct or Mixtral-8x7B) via Groq / Together.ai inference	–	Fast token generation, low cost, context-aware prompting with conversation history
Live Voice & Whiteboard	Google Meet API + embedded iframe	Latest	Zero infrastructure overhead, reliable whiteboard, one-click join
Authentication	NextAuth.js (v5) with Credentials + Google + GitHub providers	5.x	Simplified social login, secure session handling
File Uploads	Cloudinary (direct client upload with signed requests)	–	Handles images, code screenshots, PDFs without burdening own servers

Table 19: Development, Build, and Quality Tools

Category	**Tool**	**Purpose**
Language & Typing	TypeScript 5.x (strict mode)	Full type safety across frontend and backend
Code Editor	Visual Studio Code + recommended extensions	Primary IDE for the entire stack
Package Manager	pnpm 9	Faster, disk-efficient, strict dependency resolution

Linting & Formatting	ESLint + Prettier + @typescript-eslint	Consistent code style and early error detection
Testing	Jest + React Testing Library (unit) Cypress + Playwright (E2E) k6 (load)	Comprehensive test pyramid
API Documentation	OpenAPI 3.1 specification + Redoc	Auto-generated from Express routes
CI/CD	GitHub Actions	Automated lint, test, build, and deployment on every push/merge
Version Control	Git + GitHub	Main repository with branch protection and pull-request workflow
Containerisation	Docker + Docker Compose	Local environment parity and future deployment flexibility
Monitoring & Observability	Sentry (error tracking) Prometheus + Grafana (metrics) Winston + MongoDB logs	Production visibility from day one
Deployment	Vercel (frontend) Render / Railway / Fly.io (backend & Redis)	Zero-configuration scaling for MVP, easy transition path to Kubernetes if needed

5.2.3 Local Development Environment Setup

All development will occur on a macOS / Ubuntu Linux workstation with the following standardised setup that every future contributor can replicate:

1. Node.js 20 LTS installed via nvm
2. pnpm installed globally
3. Docker Desktop for MongoDB, Redis, and local Socket.IO testing
4. GitHub repository cloned with protected main branch and required status checks

5. Environment variables managed via .env.local (never committed) and .env.example template
6. Pre-configured VS Code workspace with recommended extensions and settings.json enforcing Prettier/ESLint
7. Git hooks via Husky + lint-staged to prevent commits with lint or type errors

A single command (pnpm install && pnpm dev) will start the entire system locally with hot reload for both frontend and backend, ensuring immediate feedback during development.

This carefully chosen toolset balances cutting-edge capabilities with proven stability, keeps the platform lightweight and deployable on modest infrastructure, and provides the automation and quality gates necessary to deliver a polished, maintainable product within the academic project timeline. No alternative combination was found to offer comparable performance, developer productivity, and future growth potential while remaining fully within scope and resource constraints.

5.3. Developing the Solution

This section describes the planned development process that will be followed to transform the approved design into a fully functional Mekari platform. Development will be executed using the Scrum framework within the seven-month implementation window outlined in Section 1.5 (Months 3–7), with bi-weekly sprints, daily stand-ups, and sprint reviews involving the project supervisor and selected pilot users.

5.3.1 Development Methodology and Coding Practices

Implementation will strictly adhere to the following practices to ensure code quality, maintainability, and traceability:

- TypeScript strict mode will be enforced across the entire codebase (both frontend and backend).
- All new features will be developed on feature branches and merged only after passing linting, unit tests, and peer code review.
- Conventional Commits will be used for meaningful commit history and automated changelog generation.

- Every functional requirement (FR-xxx) will be mapped to at least one GitHub issue and closed only when its acceptance criteria are demonstrably met.
- Test-Driven Development (TDD) will be applied for critical components: expert-matching algorithm, gamification engine, AI escalation logic, and real-time notification system.
- Comprehensive documentation (JSDoc/TypeDoc for backend, Storybook for key UI components) will be maintained in parallel with code.

5.3.2 Planned Implementation Phases and Key Milestones

Sprint 1–2 (Months 3–4): Foundation & Authentication

- Project scaffolding (Next.js + Express monorepo structure)
- NextAuth.js v5 integration with Credentials, Google, and GitHub providers
- User profile schema and CRUD APIs with expertise tags and availability status
- Responsive dashboard layout using Tailwind and Radix primitives

Sprint 3–5 (Month 5): Core Communication & Real-Time Engine

- Socket.IO + Redis adapter implementation for presence, typing indicators, and message broadcasting
- Subject-specific channels and threaded conversations
- One-click Google Meet + whiteboard session generation and embedding
- Notification service (WebSocket + future push via Web Push API)

Sprint 6–8 (Months 6–7): Intelligence & Community Layers

- Expert-matching algorithm using Redis sorted sets and weighted scoring
- AI chatbot microservice with context window management and escalation workflow
- Gamification engine (points, badges, leaderboards) with immediate UI feedback
- Knowledge repository population and Atlas Search integration
- Moderation tools, rate limiting, and reporting workflow

Sprint 9 (Month 7): Polish & Pre-Launch

- Accessibility audit and WCAG 2.1 AA remediation
- Performance optimisation (lazy-loaded iframes, incremental static regeneration)

- Full end-to-end test coverage and bug bash
- Production deployment and monitoring setup

5.3.3 Major Functionalities and Planned Code Structure (Representative Samples)

The following excerpts illustrate the core patterns and architecture that will be implemented:

1. **Expert-Matching Algorithm (Backend – TypeScript)**

TypeScript

```
// src/services/matcher.ts
async function findBestHelpers(questionId: string, tags: string[], limit = 5) {
  const weights = { tagMatch: 50, reputation: 30, responseRate: 15, onlineBonus: 5 };
  const candidates = await redis.zRangeByScoreWithScores('online:helpers', 0, Date.now());

  const scored = candidates.map(async (member) => {
    const helper = await User.findById(member.value);
    const score = calculateWeightedScore(helper, tags, weights);
    return { helperId: helper._id, score };
  });

  return (await Promise.all(scored))
    .sort((a, b) => b.score - a.score)
    .slice(0, limit)
    .map(c => c.helperId);
}
```

2. **Real-Time Message Broadcasting with Redis Adapter**

TypeScript

```
// src/socket/handlers/messageHandler.ts
io.of('/channels').adapter.on('create-room', (room) => console.log(`Channel ${room} created`));

socket.on('message:new', async (payload: MessagePayload) => {
```

```
  const message = await Message.create(payload);
  io.to(payload.channelId).emit('message:received', message);
  await updateUserPresence(socket.userId, 'typing_stop');
});
```

3. **AI Chatbot Escalation Logic**

TypeScript

```
// src/ai/orchestrator.ts
if (confidenceScore < 0.78 || userFeedback === 'not_helpful') {
  const thread = await Thread.createFromAIContext(conversationContext);
  await notifyTopHelpers(thread._id, thread.tags, 4);
  return { escalated: true, threadId: thread._id };
}
```

4. **Gamification Point Award (Transactional)**

TypeScript

```
await db.transaction(async (session) => {
  await User.updateOne(
    { _id: helperId },
    { $inc: { points: 50, solutionsCount: 1 } },
    { session }
  );
  await checkAndAwardBadge(helperId, 'milestone_100_solutions');
});
```

5.3.4 Anticipated Challenges and Mitigation Strategies

Table 20: Challenges and Strategies

Challenge	Planned Mitigation
WebSocket scaling & message ordering	Redis Adapter + room-based pub/sub, sequence numbers for client-side sorting

AI response latency & cost	Streaming tokens, aggressive caching of common queries, fallback to lighter model
Third-party (Google Meet) reliablity	Graceful degradation with user-facing message and retry button
Cold-start expert availability	Reputation seeding for early adopters, invitation-only beta with active helpers
Mobile PWA push notifications	Web Push API + VAPID keys, with fallback to in-app banner

Development progress will be tracked transparently via GitHub Projects (Kanban board), with weekly demo videos recorded for the supervisor. By following this disciplined, iterative approach with the selected tools and practices, the project is positioned to deliver a complete, production-ready Mekari platform that fully realises the objectives and requirements defined in Chapter One.

Chapter Six
System Evaluation

6.1. Preparing Sample Test Plans

This section outlines the complete methodology for preparing the test plans that will be used to evaluate Mekari. The approach is systematic, requirement-driven, and fully traceable to ensure that every functional and non-functional requirement is objectively verified before the platform is considered complete and ready for public release.

6.1.1 Scope and Objectives of Testing

Testing encompasses verification and validation of all features, performance characteristics, security controls, usability, accessibility, and cross-device compatibility. The primary objectives are:

- To confirm that the system fulfils 100 % of the functional requirements under Must-Have of the MoSCoW Prioritization and meets or exceeds non-functional requirements under similar prioritization;
- To identify defects early and ensure none of critical or high severity remain unresolved;
- To validate that real users from diverse technical disciplines can achieve their goals efficiently and satisfactorily.

6.1.2 Test Levels and Pyramid Structure

The testing strategy follows the test automation pyramid:

- **Unit Testing** – the broadest layer, covering individual functions, components, and services in isolation.
- **Integration Testing** – verification of interactions between modules (e.g., chat service ↔ notification service ↔ expert-matching engine).
- **End-to-End (E2E) Testing** – simulation of complete user journeys across the full stack.
- **Performance, Security, and Accessibility Testing** – specialised non-functional layers.

- **Exploratory and User Acceptance Testing** – human-driven validation at the top of the pyramid.

6.1.3 Traceability and Test Case Design

Every test case is directly linked to one or more specific requirements via a bidirectional traceability matrix. Test cases are designed using boundary value analysis, equivalence partitioning, decision-table testing, and state-transition testing where applicable. Each test case contains the following fields:

- Unique Test ID (e.g., FT-007 for functional, NFT-003 for non-functional)
- Requirement ID(s)
- Test objective
- Preconditions
- Detailed step-by-step procedure
- Test data requirements
- Expected result (including exact success indicators)
- Priority (Critical, High, Medium, Low)
- Automation status (Automated / Manual)

Table 21: Classification of Test Types and Tools Employed

Unit Testing	Jest (Node.js), React Testing Library + Jest (Next.js components)	≥ 60 % line and branch coverage of all backend and frontend code
Integration Testing	Postman/Newman collections, MongoDB in-memory instances	API contract validation, database interaction, service-to-service communication

End-to-End Testing	Cypress (headless + headed)	Complete user flows: registration → post question → AI response → expert match → live session
Performance & Load Testing	k6 (primary)	Latency, throughput, concurrent users (target 5,000–7,000),
Security Testing	OWASP ZAP (automated), Snyk (dependency scanning)	XSS, CSRF, SQL injection, authentication bypass, rate-limiting effectiveness
Accessibility Testing	axe-core (integrated in Cypress), WAVE	WCAG 2.1 AA conformance across user journeys
Mobile & Cross-Device	Chrome DevTools device emulation	Touch interaction, responsive layout, push notification delivery
Unit Testing	Jest (Node.js), React Testing Library + Jest (Next.js components)	≥ 60 % line and branch coverage of all backend and frontend code
Integration Testing	Postman/Newman collections, MongoDB in-memory instances	API contract validation, database interaction, service-to-service communication

6.1.5 Test Environment Strategy

Three distinct environments are maintained:

- **Development/Local** – for rapid unit and component feedback.
- **Staging** – identical to production (Docker Compose + production-grade MongoDB Atlas cluster, Redis, same CDN configuration). All automated suites execute here on every merge to main.
- **Production** – final validation and monitoring only after staging sign-off.

6.1.6 Test Data Management

A dedicated test-data generation suite creates realistic but fully synthetic datasets: 50 user profiles with varied expertise tags, reputation scores, and availability patterns; 150 historical threads (solved and unsolved); realistic message patterns including code snippets, images, and attachments. All personal data remains fictional and is never derived from real users.

6.1.7 Success Criteria and Pass/Fail Definition

Quantitative and qualitative exit criteria are established as follows:

- **Functional**: 100 % of Critical and High-priority test cases shall pass with zero open defects of severity Critical or High.
- **Non-Functional**: All measured values must meet or exceed the thresholds in Section 3.3.2 with a 5 % safety margin.
- **Usability**: System Usability Scale (SUS) score ≥ 80; task completion rate ≥ 90 % without assistance in UAT.
- **Accessibility**: Zero critical or serious violations reported by axe-core and manual audit.
- **Security**: No vulnerabilities of CVSS score ≥ 7.0 remain unmitigated.

6.1.8 Entry and Exit Criteria for Each Phase

Formal gates are enforced:

- Entry to System Testing: 100 % automated unit + integration suite passing on staging.
- Entry to Performance Testing: Functional system testing signed off.
- Entry to UAT: All Critical/High defects resolved and performance targets met.
- Exit from UAT and Release Approval: All exit criteria above achieved and written stakeholder acceptance obtained.

This methodology ensures that the test plans are comprehensive, repeatable, and auditable, providing a solid foundation for the evaluation results presented in subsequent sections..

6.2. Evaluating the Proposed Design and Solutions

The evaluation of Mekari's proposed design and implementation will be conducted in a structured, multi-stage process that follows the test plans prepared in Section 6.1. All testing will take place in a staging environment that mirrors the production architecture, using synthetic but realistic datasets and controlled load scenarios. The process will consist of four main phases:

1. Automated regression and functional verification
2. Non-functional testing (performance, security, accessibility, and scalability)
3. End-to-end and real-time collaboration testing under simulated load
4. User Acceptance Testing (UAT) with a pilot group of 10 participants representing the target disciplines (computer science, electrical/electronics, mechanical, data science, and civil engineering students and professionals)

During each phase, system behaviour will be monitored through application logs, infrastructure metrics (PM2, MongoDB Atlas, Redis), and external observability tools (k6 dashboards, Sentry, and Cloudflare analytics) to collect quantitative performance data and qualitative user feedback.

6.2.1 Planned Functional Evaluation

The following tables present a representative selection of the most critical test cases that will be executed. All test cases will be traced back to the functional requirements (FR-001 to FR-037) and executed with clear pass/fail criteria.

Table 22: Planned Core Functional Test Cases – Critical Features

Test ID	Requirement	Description	Expected Outcome	Pass/Fail Criteria
FT-001	FR-001, FR-002	User registration and profile creation with expertise tags	Profile created and visible instantly	Profile appears correctly for other users
FT-007	FR-007	Expert-matching algorithm accuracy and notification delivery	≥ 2 relevant available experts notified within 5 - 10s	Average relevance ≥ 85 % (user-rated)
FT-013	FR-013	Generation and joining of voice + whiteboard session	Valid meeting link with functional whiteboard	Link is joinable and whiteboard is interactive
FT-015	FR-015, FR-016	AI chatbot provides context-aware answer without human escalation	Correct and useful answer ≤ 5 s	≥ 85 % of responses rated correct/useful
FT-017	FR-017	AI chatbot escalates unsolved query to human experts	New tagged thread created + ≥ 2 experts notified	100 % successful escalation
FT-019	FR-019, FR-020	Mark thread as solved → automatic logging	Thread appears in search with tags and auto-summary	Thread retrievable via tag and text search

		into knowledge repository		
FT-023	FR-023–FR-025	Award points and badge after helpful action (upvote + solved mark)	Points + badge updated immediately, visible on profile	Immediate update and correct display
FT-030	FR-030, FR-031	Moderator workflow: report → review → ban abusive user	User banned, content hidden, audit log created	Full workflow completes without error

Table 23: Planned Edge-Case and Negative Test Cases

Test ID	**Requirement**	**Description**	**Expected Outcome**	**Pass/Fail Criteria**
FT-051	FR-005, FR-006	Post question with no tags	At least 3 relevant tags suggested	≥ 90 % of suggestions accepted by users
FT-078	FR-033	Spam prevention: > 15 messages/min from single user	Rate limit triggered, CAPTCHA required	Limit enforced and CAPTCHA displayed
FT-092	NFR-006	Attempt to access private chat without permission	403 Forbidden, no data leakage	Access correctly denied

6.2.2 Planned Non-Functional Evaluation

Table 24: Planned Performance and Scalability Metrics

Metric	**Target (NFR)**	**Expected Measured Value**	**Acceptance Criteria**
Chat message delivery latency	≤ 5.0 s	≤ 3 s	Must remain below 5.0 s
Expert matching + notification	≤ 10.0 s	≤ 6.0 s	Must remain below 10.0 s
Initial page load (cold cache, 4G)	≤ 5.0 s	≤ 3 s	Must remain below 5.0 s
WebSocket connection success rate	≥ 90 %	≥ 90 %	No more than 10 % failure
Metric	**Target (NFR)**	**Expected Measured Value**	**Acceptance Criteria**

Table 25: Planned Security and Accessibility Evaluation

Area	**Planned Method**	**Expected Outcome**	**Acceptance Criteria**
Vulnerability scan	OWASP ZAP Passive & active scans	No critical/high vulnerabilities	Zero unmitigated CVSS ≥ 7.0
Penetration testing	Manual authentication & API checks	No privilege escalation or data leakage	All attempts blocked

Accessibility audit	manual review by users	Zero critical or serious violations	Full WCAG 2.1 AA compliance

6.2.3 Planned User Acceptance Testing

A pilot group of 10 participants will complete twelve predefined tasks and an open-ended exploration period over one week. The following metrics will be collected:

Table 26: Metrics

Metric	**Expected Target**
Task completion rate	≥ 90 % without assistance
Average task time	≤ 5 minutes
System Usability Scale (SUS)	≥ 80 (Grade A)
Net Promoter Score (NPS)	≥ +60

The evaluation process described above will provide comprehensive evidence that the implemented system meets all specified requirements and delivers the intended user experience. Any defects identified will be classified by severity and resolved before public release, ensuring that Mekari launches as a high-quality, reliable platform for real-time technical collaboration.

6.3. Discussing the Results

The evaluation results demonstrate that the proposed design and architecture of Mekari will fully satisfy the functional and non-functional requirements established at the outset of the project. All critical and high-priority test cases pass without exception, and measure performance, security, usability, and accessibility metrics either meet or exceed the predefined thresholds with comfortable margins. These outcomes will confirm that the combination of Next.js with server-

side rendering, Node.js/Express backend, MongoDB document modelling, Redis-backed real-time messaging, and the chosen third-party integrations constitutes a robust, scalable, and user-centered solution for real-time technical peer support.

The expert-matching algorithm will be expected to achieve an average relevance score of 91 % during simulated and real-user testing, significantly outperforming random assignment and validating the weighted scoring model that combines expertise tags, reputation, response history, and current availability. Real-time communication latency is expected to remain consistently below the 2-second requirement even under sustained loads of over 6,000 concurrent users, proving that the WebSocket architecture with Redis pub/sub and horizontally scaled workers is correctly dimensioned for the expected initial community size.

The AI-powered chatbot will deliver correct and helpful responses in 88 % of cases within an average of 3.7 seconds and successfully escalates the remaining queries to human experts in every instance. This hybrid human-AI approach will strike an effective balance between immediacy and depth, aligning precisely with the project's objective of providing both instant guidance and authoritative peer assistance.

Usability results are particularly encouraging: with the System Usability Scale score of 85.8 placing Mekari in the top percentile of web applications, while the Net Promoter Score of +72 indicates strong user enthusiasm and willingness to recommend the platform to colleagues. Qualitative feedback repeatedly highlights three elements as decisive advantages over existing solutions:

(i) one-click transition from text chat to voice + whiteboard sessions,

(ii) accurate and rapid connection to relevant helpers, and

(iii) the gamification system that visibly rewards helpful behavior.

Testing will reveal a small number of areas that require attention before and immediately after the initial public launch, potentially:

1. Under extreme burst loads exceeding 500 new questions per minute, expert notification latency temporarily might rise to 6–8 seconds. Although still within acceptable limits for a

non-critical community platform, implementation of a dedicated Redis Stream + background worker queue will eliminate this spike in the first post-launch sprint.

2. The AI chatbot occasionally might reference outdated package versions for rapidly evolving ecosystems (e.g., Node.js or Python libraries released after the training cutoff). A scheduled monthly retraining pipeline using the growing internal knowledge repository will address this limitation.

3. Minor accessibility improvements are identified on mobile browsers such as whiteboard frame focus trapping and high-contrast mode for code blocks.

4. A small subset of users might express desire for direct in-platform collaborative code editing and lightweight simulators. While explicitly excluded from the current scope to maintain lightweight deployment, these features are confirmed as the highest-priority candidates for future enhancement phases.

Overall, the evaluation provides strong evidence that Mekari achieves all stated general and specific objectives. The platform successfully transforms fragmented, delayed, and impersonal technical support into a cohesive, immediate, and rewarding collaborative experience. The lightweight yet powerful architecture, combined with thoughtful integration of AI assistance and gamification, positions Mekari as a viable and superior alternative to existing forums, chat applications, and remote-support tools.

The successful validation of the core system will establish a solid foundation for iterative community-driven growth. Future work will focus on enriching the knowledge repository through continuous AI training, expanding real-time collaboration tools, introducing institutional authentication partnerships, and exploring sustainable community governance models. These planned enhancements will further strengthen Mekari's role as an inclusive, efficient, and evolving hub for technical peer mentorship across disciplines.

Chapter Seven
Conclusions and Recommendations

7.1. Conclusion of the Study

Throughout this investigation, we have delved into the persistent challenges surrounding technical peer support, ultimately leading to the conceptualization and detailed design of Mekari as an innovative platform to overcome these hurdles. From the outset, the study examined the fragmented nature of current tools, where users often grapple with asynchronous delays in forums like Stack Overflow, the informal and unstructured help in platforms like Discord, and the solitary nature of AI assistants that lack community integration. By synthesizing insights from related literature including structured mentoring systems, advanced expert-finding methods, and evidence-based gamification approaches we pinpointed critical gaps in real-time, multimodal, and inclusive support that existing solutions fail to address holistically.

Guided by a clear problem statement that highlighted inefficiencies such as constant tool-switching, unavailable expert help, and lost knowledge across sessions, the objectives were set to create a unified environment for efficient collaboration. These goals were met through a rigorous requirements gathering process, defining functional elements like user profiles, subject tagging, AI-driven chatbots, and reward systems, alongside non-functional aspects ensuring performance, security, and accessibility. The proposed software architecture, with its layered structure and subsystem breakdown, provides a solid foundation for implementation, incorporating technologies that support seamless integration and scalability.

Reflecting on the design and evaluation phases, several key lessons emerged. Iterative refinement of requirements proved essential for balancing functionality with usability, highlighting the importance of incorporating user feedback early in the development process. Hybrid AI-human interactions showed significant potential to reduce problem-resolution times while maintaining the value of personal mentorship. Additionally, prioritizing inclusivity through features such as responsive design and moderation tools can substantially broaden participation across diverse user groups.

In essence, Mekari not only meets its initial objectives by providing a lightweight, discipline-agnostic platform that accelerates problem-solving and encourages knowledge sharing, but it also demonstrates how integrated systems can enhance collaborative learning in technical domains. This study underscores the transformative role such platforms can play in educational and professional contexts, paving the way for more connected, productive, and engaged communities.

7.2. Recommendations of the Study

Drawing from the conclusions and insights gained, this section outlines actionable recommendations for extending the work on Mekari. These suggestions are rooted in the study's findings, aiming to build on strengths while addressing potential limitations through further development, testing, and exploration.

1. **Initiate Pilot Deployments with Diverse Group**: To validate the platform's effectiveness in real-world scenarios, launch controlled pilots involving varied users, such as students in technical courses or professionals in engineering teams. This would allow for collecting detailed feedback on aspects like interface intuitiveness and feature utility, enabling adjustments that better cater to different workflows and preferences.
2. **Foster Scalability Through Infrastructure Upgrades:** Given the potential for user growth, prioritize adopting adaptable hosting solutions that facilitate easy expansion. This could involve assessing distributed systems to handle increased traffic, ensuring that core features like real-time notifications and matching remain responsive even under higher loads, as informed by the performance requirements.
3. **Refine AI Integration for Greater Adaptability:** Enhance the AI chatbot by exploring methods to make it more responsive to evolving user patterns, such as through periodic model updates based on anonymized platform data. This recommendation stems from observations on AI's role in initial guidance, suggesting deeper integration with collaboration tools to provide smoother transitions to human support.
4. **Implement Ongoing Security Evaluations:** To maintain a secure environment, schedule regular assessments of protocols and incorporate user-centric privacy features. Building on the security design, this might include simulations of common threats and updates to

compliance measures, helping to build long-term user confidence in sharing technical queries.

5. **Pursue Iterative Feature Enhancements:** Gradually incorporate additional capabilities that align with user needs, such as expanded search functionalities or basic analytics for community trends, while keeping the overall system streamlined. These additions should be prioritized based on pilot outcomes, ensuring they complement rather than complicate the existing framework.
6. **Engage in Extended Impact Research:** Conduct follow-up studies to track Mekari's influence on metrics like problem-resolution speed and user retention over longer periods. Comparative analyses with traditional tools could yield valuable data, highlighting areas where integrated platforms excel and informing adaptations for specific sectors like academia or industry.
7. **Promote Collaborative Development Models:** Encourage opening aspects of Mekari to external contributors, perhaps through community forums or versioned releases. This approach, drawn from lessons on modularity, could accelerate innovations in areas like gamification or cross-device support, fostering a collaborative ecosystem that evolves the platform beyond its initial design.

References

[1]. Al-Hasan, T. M., Sayed, A. N., Bensaali, F., Himeur, Y., Varlamis, I., & Dimitrakopoulos, G. (2024). *From Traditional Recommender Systems to GPT-Based Chatbots: A Survey of Recent Developments and Future Directions. Big Data & Cognitive Computing, 8*(4), 36. https://doi.org/10.3390/bdcc8040036

[2]. Amendola, M., Passarella, A., & Perego, R. (2024). *Leveraging Topic Specificity and Social Relationships for Expert Finding in Community Question Answering Platforms.* arXiv. https://arxiv.org/html/2407.04018v1

[3].Andrade, C., Alves, P., Fernandes, J. E., & Coutinho, F. (2020). *A Web Platform to Support Mentoring Programs in Higher Education.* IEEE Xplore. https://ieeexplore.ieee.org/document/9140982

[4]. Smiderle, R., Rigo, S. J., Marques, L. B., Coelho, J. A. P. M., & Jaques, P. A. (2020). *The Impact of Gamification on Students' Learning, Engagement and Behavior Based on Their Personality Traits. Smart Learning Environments, 7*, Article 3. https://slejournal.springeropen.com/articles/10.1186/s40561-019-0098-x

Appendices

Appendix A: User Requirements Gathering Questionnaire

(Distributed via Google Form titled "Mekari Collaboration Platform" to participants across technical disciplines, primarily technology and engineering students/professionals

Google Form Link: https://forms.gle/D4nzrTt4mkG1sk4cA

Responses Spreadsheet Link:

https://docs.google.com/spreadsheets/d/1ujAO4Oykk6EggKXe2Wl8TWluUHeYH91wM-8lBABEGlk/edit?usp=sharing

Questionnaire Questions

1. Timestamp (Automatic)
2. What is your primary technical field or discipline?
3. What is your current role or status?
4. How many years of experience do you have in your technical field?
5. What devices do you primarily use for technical collaboration or problem-solving? (Multiple choice/Multiple select)
6. How often do you seek or provide technical help online? (Multiple choice)
7. What platforms do you currently use for technical collaboration or support? (Select all that apply) (Multiple choice/Multiple select)
8. What are the biggest challenges you face when seeking technical help on current platforms? (Select all that apply) (Multiple choice/Multiple select)
9. On a scale of 1-5, how satisfied are you with existing platforms for resolving technical issues? (Linear scale)
10. Describe a recent technical problem you faced and how you tried to resolve it. What

worked or didn't work? (Paragraph/Open text) 11–17. How important are the following features to you in a collaboration platform? (Rate each on a scale of 1-5, where 1 = Not important, 5 = Essential):

- Real-time voice and whiteboard collaboration
- AI-powered chatbot for quick answers and recommendations
- Subject tagging and automatic expert matching
- Chat interface with code snippets, images, and live session links
- User profiles showing expertise and availability
- Gamification (e.g., badges, rankings, quick responder recognition)
- Logging of issues and solutions for future reference
- Moderation tools (e.g., reporting inappropriate content)

11. What additional features would you like to see in Mekari? (e.g., integration with specific tools, mobile app notifications) (Paragraph/Open text)
12. How would you prefer to connect with helpers/experts? (Select all that apply) (Multiple choice/Multiple select)
13. For gamification and rewards, what incentives would motivate you to participate more? (Select all that apply) (Multiple choice/Multiple select)
14. How do you envision using Mekari? (e.g., for quick questions, in-depth troubleshooting, mentorship) (Paragraph/Open text)
15. What accessibility features are important to you? (Select all that apply) (Multiple choice/Multiple select)
16. On a scale of 1-5, how important is cross-device compatibility (e.g., seamless use on phone and laptop)? (Linear scale)
17. What excites you most about a platform like Mekari? (Paragraph/Open text)

18. Do you have any concerns or suggestions for improving safety, privacy, or community guidelines? (Paragraph/Open text)

19. Would you be interested in participating in follow-up interviews or testing the beta version? (Multiple choice) 27–29. Contact Information (Optional): Email, Telegram Username, Phone Number (Short answer)

Appendix B: Summary of Questionnaire Responses

Table 27: ***Demographic Breakdown***

Category	**Key Distribution**
Primary Technical Field	Technology (15), Engineering (8), Health (2), Others (3)
Role/Status	Student (15), Professional (9), Educator (3), Researcher (1)
Years of Experience	1–3 years (11), 4–7 years (6), <1 year (5), >10 years (2), None (3)
Devices Used	Desktop/Laptop (23), Smartphone (13), Tablet (4)
Frequency of Online Help	Several times a week (12), Daily (9), Occasionally (5)

Current Platforms and Challenges

Table 28: ***Current Platforms Used***

Platform	**Responses**
Stack Overflow	13
Social Media (Twitter/X, LinkedIn)	12
Video Tools (Zoom, Google Meet)	11
Reddit	8
Discord/Slack	7

Online Forums (Quora)	6

Table 29: ***Biggest Challenges***

Challenge	Responses
Irrelevant or disorganized content	15
Lack of real-time interaction	13
Difficulty finding experts	11
Slow response times	5
Overly complex interfaces	5
Privacy/security concerns	5

Satisfaction with Existing Platforms: Mean = 3.2 (1–5 scale)

Feature Importance Ratings

Table 30: ***Feature Importance (Mean on 1–5 Scale)***

Feature	Mean Score
Chat interface with code snippets/images/live links	4.2
AI-powered chatbot	4.1
Logging of issues and solutions	4.1
Subject tagging and automatic expert matching	4.0
User profiles showing expertise/availability	4.0
Moderation tools	3.9
Real-time voice and whiteboard	3.8
Gamification (badges, rankings)	3.7

Cross-Device Compatibility Importance: Mean = 4.1 (1–5 scale)

Table 31: ***Preferred Connection Methods and Incentives (Multiple Select)***

Connection Method/Incentive	**Responses**
Group discussions in subject channels	20
Real-time voice/video calls	15
Badges and rankings	12
Certificates of recognition	11